

POSTS AND TELECOMMUNICATIONS INSTITUTE OF
TECHNOLOGY
Faculty of Information Technology

Intelligent System Development
ASSIGNMENT 05

Convolutional Neural Networks:
CNN-3 vs. CNN-5 in TensorFlow/Keras and PyTorch

"Same architecture, same data: depth changes the result, the framework does not."

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Email: quetd@ptit.edu.vn, tdque@yahoo.com

Student Name: Lưu Anh Dũng
Student ID: B23DCDK036
Semester: I.2025 – 2026
Class: E23CNPM02
Submission: Individual Project

Hanoi, 2026

Contents

1	Executive Summary	2
2	CNN Fundamentals	2
2.1	Convolution	3
2.2	ReLU, pooling, batch normalization	3
2.3	Output, loss and training	3
2.4	One image through CNN-3, step by step	3
2.5	Learning: one backpropagation step by hand	5
2.6	One epoch of training	6
2.7	Training mode vs. evaluation mode	6
2.8	1-D convolution on a table	6
2.9	Metrics	6
3	Project Structure and Setup	7
4	Datasets	7
4.1	SVHN	7
4.2	GTSRB	8
4.3	Diabetes (Playground Series S5E12)	9
5	Models	10
5.1	TensorFlow, Keras and PyTorch	11
5.2	Implementation (TensorFlow/Keras)	11
5.3	Implementation (PyTorch)	12
6	Results per Dataset	13
6.1	SVHN	13
6.2	GTSRB	15
6.3	Diabetes	17
7	Cross-Dataset Comparison	19
7.1	Keras vs. PyTorch outputs	20
7.2	Why depth helps on images: the receptive field	24
7.3	Why a CNN does not help on the diabetes table	25
8	Conclusion and Limitations	25
A	Appendix	26
A.1	Additional figures	26
A.2	How to run the project	29

1 Executive Summary

This assignment builds convolutional neural networks with **3** and **5** convolutional layers (CNN-3, CNN-5) in **TensorFlow/Keras** (TF) and **PyTorch** (PT), and trains them on three datasets: **SVHN** (99,289 house-number photos, 10 classes), **GTSRB** (51,839 traffic-sign photos, 43 classes) and a **diabetes** table (300,000 patients, 24 features, binary target). Every pair of runs uses the same split, architecture, parameter count, optimizer, batch size, early-stopping rule and seed, so depth and framework are the only variables. Each model has a run ID `<dataset>-<framework>-<model>`, e.g. `SV-PT-CNN-5`.

Table 1: All 12 runs (test set). s/epoch = median training time per epoch, CPU.

Run ID	Dataset	Fw.	Model	Params	Epochs	Accuracy	Macro F1	s/epoch
SV-TF-CNN-3	SVHN	TF	CNN-3	129,290	20	0.9117	0.9072	45.0
SV-TF-CNN-5	SVHN	TF	CNN-5	175,658	10	0.9306	0.9265	86.5
SV-PT-CNN-3	SVHN	PT	CNN-3	129,290	18	0.9106	0.9046	48.9
SV-PT-CNN-5	SVHN	PT	CNN-5	175,658	13	0.9294	0.9250	98.7
GT-TF-CNN-3	GTSRB	TF	CNN-3	137,771	20	0.9781	0.9751	23.3
GT-TF-CNN-5	GTSRB	TF	CNN-5	184,139	11	0.9911	0.9907	45.5
GT-PT-CNN-3	GTSRB	PT	CNN-3	137,771	20	0.9848	0.9843	25.9
GT-PT-CNN-5	GTSRB	PT	CNN-5	184,139	16	0.9943	0.9937	52.8
DB-TF-CNN-3	Diabetes	TF	CNN-3	191,169	7	0.6465	0.6310	12.8
DB-TF-CNN-5	Diabetes	TF	CNN-5	391,329	5	0.6333	0.6259	28.8
DB-PT-CNN-3	Diabetes	PT	CNN-3	191,169	7	0.6371	0.6280	20.1
DB-PT-CNN-5	Diabetes	PT	CNN-5	391,329	8	0.6446	0.6308	42.5

Key takeaways

1. **Depth helps on images.** CNN-5 beats CNN-3 in both frameworks: +0.019 accuracy on SVHN, +0.009 to +0.016 macro F1 on GTSRB. The best model is `GT-PT-CNN-5` (macro F1 0.9937).
2. **Depth does not help on the table.** On diabetes all four CNNs reach macro F1 0.626–0.631 and ROC-AUC 0.694–0.696; an MLP with 6,977 parameters (vs. 391,329) scores ROC-AUC 0.695. The limit is the weak signal in the features, not model capacity.
3. **Same model, same result.** With identical settings, TF and PT differ by at most 0.011 accuracy on every dataset; the parameter counts are identical (checked in code).
4. **Depth costs time.** CNN-5 needs 1.9–2.2× the time per epoch of CNN-3. On this CPU run, PyTorch was 9–16% slower per epoch than Keras on images and about 50% slower on the table.

2 CNN Fundamentals

Terms are defined once here and used with the same meaning in the rest of the report. Notation: N samples; input $C \times H \times W$ (channels, height, width); kernel size K , stride S , padding P ; F filters (output channels); logits z , predicted probability p , true label y .

2.1 Convolution

A convolutional layer slides F learned **filters** of size $K \times K \times C_{in}$ over its input. At each position it computes one value of a **feature map**:

$$\text{out}(i, j) = \sum_{u, v, c} \text{window}(i+u, j+v, c) \cdot w(u, v, c) + b, \quad H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1.$$

Example (SVHN notebook, §4.2). A 5×5 patch containing a vertical stroke (columns 0 0 1 1 0) and the vertical-edge filter with rows $[-1, 0, 1]$: window $[0, 0, 1]$ gives $(-1 \cdot 0 + 0 \cdot 0 + 1 \cdot 1) \times 3 = 3$ (left edge of the stroke), window $[1, 1, 0]$ gives -3 (right edge). The notebook checks this against the framework’s convolution. With $K = 3$, $S = 1$, **padding “same”** ($P = 1$) keeps $32 \times 32 \rightarrow 32 \times 32$.

2.2 ReLU, pooling, batch normalization

Table 2: Layer operations with worked examples (from the notebooks).

Layer	Computation	Example
ReLU	$\max(0, x)$	$-3 \rightarrow 0, 3 \rightarrow 3, -1.3 \rightarrow 0$
Max pooling 2×2	largest value of each 2×2 block; halves H, W ; no parameters	$[[0.1, 0.9], [0.4, 0.2]] \rightarrow 0.9$
Batch normalization	$\hat{x} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$, out = $\gamma \hat{x} + \beta$, per channel over the batch	$[1, 3, 5] \rightarrow [-1.22, 0, 1.22]$; dark/bright pixels $[0.05, 0.10, 0.80, 0.90] \rightarrow [-1.06, -0.93, 0.87, 1.12]$
Global average pooling	mean of each feature map	$4 \times 4 \times 128 \rightarrow 128$
Dropout (0.3)	in training, 30% of values set to 0	reduces overfitting

Receptive field = input area that can influence one output unit. It grows as $RF \leftarrow RF + (K - 1) \cdot j$ per layer, where the jump j doubles after each pooling: 22×22 pixels for CNN-3 and 28×28 for CNN-5 (of a 32×32 image).

Parameters. Conv: $(K \cdot K \cdot C_{in} + 1) \cdot F$, e.g. first layer $(3 \cdot 3 \cdot 3 + 1) \cdot 32 = 896$; BatchNorm: $2F$ trainable (γ, β) ; Dense: $(\text{inputs} + 1) \cdot \text{units}$.

2.3 Output, loss and training

- **Softmax** (multi-class): $p_k = e^{z_k} / \sum_j e^{z_j}$. **Cross-entropy**: $L = -\ln p_{\text{true}}$. Example: logits “3” = 2, “8” = 1, others 0 $\Rightarrow p(\text{“3”}) = 0.41$, loss 0.90 if the image is a 3, 1.90 if it is an 8. An untrained model starts at $\ln 10 = 2.30$ (SVHN) or $\ln 43 = 3.76$ (GTSRB).
- **Sigmoid** (binary): $p = 1 / (1 + e^{-z})$. **Binary cross-entropy**: $L = -[y \ln p + (1 - y) \ln(1 - p)]$. Example: $z = 1.2 \Rightarrow p = 0.77$; loss 0.26 if $y = 1$, 1.47 if $y = 0$.
- **Adam**: $w \leftarrow w - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$, $\eta = 0.001$. **Epoch**: one pass over all training batches (256 samples each). **Early stopping**: stop after 3 epochs without a lower validation loss and restore the best epoch’s weights.

2.4 One image through CNN-3, step by step

Figures 1 and 2 follow one SVHN test image (a “3”) through a PyTorch CNN-3. The figures were made with a CNN-3 trained for 3 epochs only to illustrate the flow; the reported results come from the full runs in Section 6.

Table 3: What happens to one image in CNN-3 (SVHN).

Step	Output shape	What it means
input	$3 \times 32 \times 32$	3,072 numbers in $[0, 1]$: red, green and blue value of each pixel
block 1	$32 \times 16 \times 16$	32 filters each scan the image for one small pattern (edge, colour change). Each map shows <i>where</i> its pattern was found; pooling halves the size
block 2	$64 \times 8 \times 8$	filters now combine block-1 maps: curves and stroke pieces over a larger area
block 3	$128 \times 4 \times 4$	each unit sees 22×22 pixels: parts of a digit (a loop, a bar)
GAP	128	mean of each 4×4 map: “how strongly is pattern i present anywhere”
Dense 256 $\rightarrow$ 10		weighted sums of the 128 values give one score (logit) per digit
Dense 10		
softmax	10	scores $\rightarrow$ probabilities that sum to 1

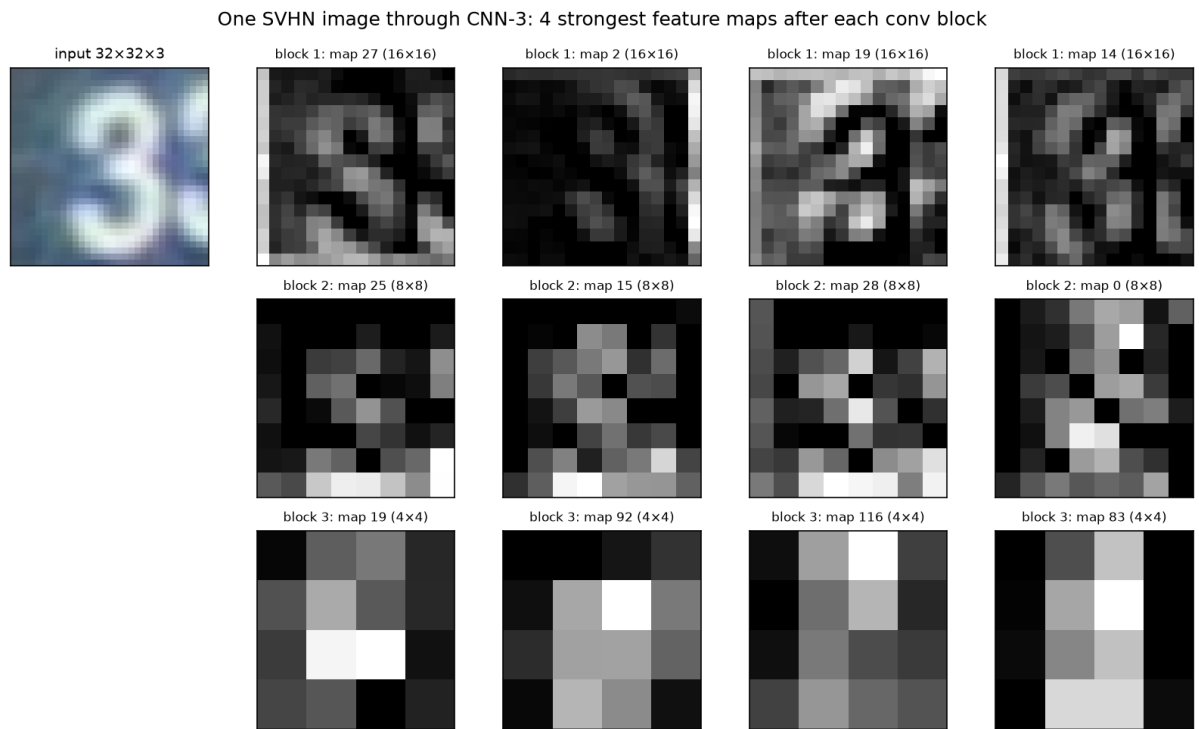


Figure 1: Input image and the 4 strongest feature maps after each conv block (bright = strong response). Maps get smaller and more abstract: block 1 still looks like the digit, block 3 only marks *where* digit parts are.

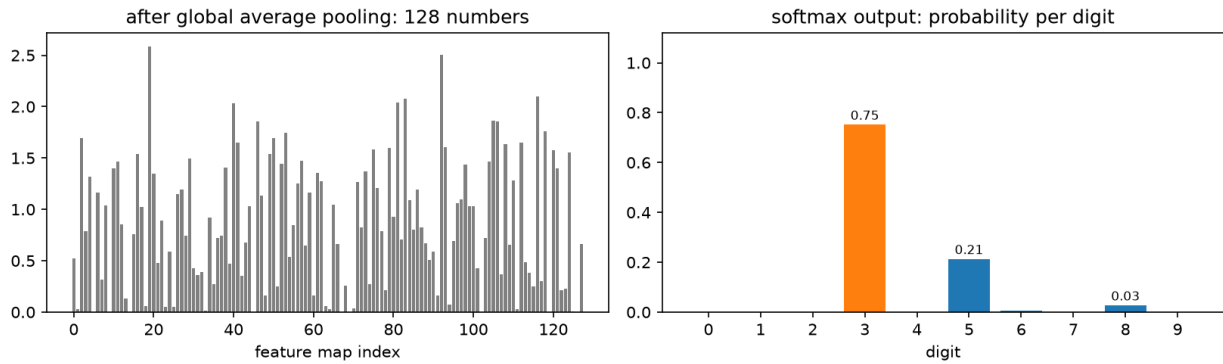


Figure 2: The 128 values after global average pooling (left) and the final probabilities (right): $p(3) = 0.75$, $p(5) = 0.21$. The runner-up “5” shares the lower curve of the “3”; this 5/3 pair is also one of the most frequent confusions in the full results.

2.5 Learning: one backpropagation step by hand

Training changes every weight a little so that the loss goes down. PyTorch does this with `loss.backward()` (compute the gradients) and `optimizer.step()` (update the weights). The smallest example is one neuron with one input:

$$z = wx + b, \quad p = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad L = -[y \ln p + (1 - y) \ln(1 - p)].$$

Forward with $x = 2$, $w = 0.5$, $b = 0$, true label $y = 1$: $z = 1.0$, $p = \sigma(1.0) = 0.731$, $L = -\ln 0.731 = 0.313$.

Backward (chain rule). For sigmoid + binary cross-entropy the derivative simplifies to $\partial L / \partial z = p - y$, so

$$\frac{\partial L}{\partial z} = 0.731 - 1 = -0.269, \quad \frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w} = -0.269 \times x = -0.538, \quad \frac{\partial L}{\partial b} = -0.269.$$

Update (gradient descent, learning rate $\eta = 0.1$): $w \leftarrow w - \eta \partial L / \partial w$:

$$w = 0.5 - 0.1 \times (-0.538) = 0.554, \quad b = 0 - 0.1 \times (-0.269) = 0.027.$$

Check: with the new weights $z = 0.554 \cdot 2 + 0.027 = 1.135$, $p = 0.757$, $L = 0.279$: the loss went down from 0.313 to 0.279, and p moved towards the true label 1. The gradient was negative, so the weight went *up*.

In a CNN the same chain rule runs backwards through every layer:

Layer	How the gradient passes through
Softmax + cross-entropy	$\partial L / \partial z_k = p_k - [k = y]$, e.g. $p = (0.75, 0.21, \dots)$ for (“3”, “5”) and true “3”: -0.25 for “3”, $+0.21$ for “5”
Dense	$\partial L / \partial w_{ij} = \partial L / \partial z_j \cdot x_i$ (as in the neuron above)
ReLU	gradient $\times 1$ where the input was > 0 , $\times 0$ where it was cut to 0
Max pooling	the whole gradient goes to the one input that was the maximum; the other three get 0
Convolution	$\partial L / \partial w = \text{sum over all positions of (gradient at that output} \times \text{the input window there)}$: one filter learns from every place it was applied

Adam then scales each weight’s step by running averages of its gradients (Section 2.3), and one epoch repeats this for every batch.

2.6 One epoch of training

An **epoch** is one pass over the whole training set, in batches of 256 samples:

Step	Code (PyTorch)	What happens
1	<code>model.train()</code>	training mode: Dropout on, BatchNorm uses the batch's μ, σ
2	<code>for x, y in train_loader:</code>	a new random order of batches every epoch
3	<code>optimizer.zero_grad()</code>	clear the gradients of the previous batch
4	<code>loss = criterion(model(x), y)</code>	forward pass + loss (Sections 2.4, 2.3)
5	<code>loss.backward()</code>	gradient of the loss for every weight (Section 2.5)
6	<code>optimizer.step()</code>	Adam changes every weight a little
7	<code>model.eval() + validation</code>	loss and accuracy on the validation set, no weight changes
8	early stopping	new lowest validation loss $\rightarrow$ save the weights; 3 epochs without improvement $\rightarrow$ stop

Numbers per epoch: SVHN 69,502 training images $\rightarrow$ 272 weight updates; GTSRB 36,287 $\rightarrow$ 142; diabetes 210,000 rows $\rightarrow$ 821. **Example of early stopping (SV-PT-CNN-3):** the validation loss was lowest at epoch 15, did not improve in epochs 16–18, so training stopped after epoch 18 and the weights of epoch 15 were restored.

2.7 Training mode vs. evaluation mode

Two layers behave differently while training and while predicting, which is why the code switches between `model.train()` and `model.eval()` (Keras does this automatically inside `fit` and `predict`).

Layer	Training	Evaluation (validation, test)
BatchNorm	normalizes with the current batch's μ, σ and updates running averages: $\text{running} = 0.9 \cdot \text{running} + 0.1 \cdot \text{batch}$	uses the stored running μ, σ , so one image gives the same output whatever batch it is in
Dropout (0.3)	sets a random 30% of values to 0 and multiplies the rest by $1/0.7$ (e.g. $[1.0, 2.0, 0.5] \rightarrow [1.43, 0, 0.71]$)	does nothing: all values pass unchanged

Forgetting `model.eval()` would make test predictions random (Dropout) and batch-dependent (BatchNorm).

2.8 1-D convolution on a table

A patient is a vector of d standardized features, treated as a sequence of length d with one channel. A Conv1D filter of size 3 mixes three neighbouring features: $\text{out}[i] = w_1x[i-1] + w_2x[i] + w_3x[i+1] + b$. Example: $x = [0.5, -1.2, 0.3, 2.0, -0.4, 0.0]$, $w = [1, 0, -1] \Rightarrow \text{out} = [1.2, 0.2, -3.2, 0.7, 2.0, -0.4]$ (checked against the framework's Conv1D). Unlike pixels, table columns have no natural order, so “neighbours” depend on the chosen column order; the tabular models therefore **flatten** the last feature map instead of averaging it (position = feature identity), and an MLP baseline is trained for reference.

2.9 Metrics

Per class: precision = $TP/(TP + FP)$, recall = $TP/(TP + FN)$, $F1 = 2PR/(P + R)$. **Macro F1** averages the per-class F1 with equal weight. With unbalanced classes accuracy can mislead: on a toy test set of 95 “big class” and 5 “rare class” images, always predicting the big class gives

accuracy 0.95 but macro F1 0.49. **ROC-AUC** (binary) is the probability that a random positive sample gets a higher p than a random negative one ($0.5 =$ guessing).

3 Project Structure and Setup

Table 4: Folders and naming convention.

Item	Content / rule
dataset/sv, gt, db	raw Kaggle files (not committed) + <code>splits.npz</code> (row indices of the split)
notebook/	<code>sv_svhn.ipynb</code> , <code>gt_gtsrb.ipynb</code> , <code>db_diabetes.ipynb</code> : one self-contained notebook per dataset
report/	this report, <code>images/</code> (figures extracted from the notebooks), <code>make_summary.py</code>
Dataset / framework / model	SV, GT, DB / TF, PT / CNN-3, CNN-5 (number of conv layers)
Run ID and figure name	<dataset>-<framework>-<model>; figure <run_id>-<plot>.png

Each notebook has the same sections: setup, data, exploration, TensorFlow, PyTorch, comparison, conclusion. Every code cell is preceded by a short explanation cell (formula, worked example, shapes).

Fairness rules. Stratified 70/15/15 split saved once and shared by both frameworks; batch 256; Adam ($\eta = 10^{-3}$, $\epsilon = 10^{-7}$); at most 20 epochs; early-stopping patience 3; Glorot-uniform weights and zero biases in both frameworks (PyTorch re-initialized to match Keras); BatchNorm momentum 0.9 in Keras $\equiv$ 0.1 in PyTorch (both mean running $= 0.9 \cdot \text{running} + 0.1 \cdot \text{batch}$), $\epsilon = 10^{-5}$; the trainable parameter count of each PyTorch model is asserted equal to the Keras one.

Environment of the reported run. Windows, Python, TensorFlow 2.22 (release candidate), PyTorch 2.14. Both frameworks ran on the **CPU** (TensorFlow has no GPU support on native Windows; the PyTorch build was CPU-only), so speed comparisons are CPU vs. CPU. Time per epoch is the median of epochs 2 onwards (epoch 1 includes setup).

4 Datasets

Table 5: Datasets (all from Kaggle, download < 300 MB each).

Code	Source	Samples	Classes	Input
SV SVHN	sahityasetu/street-view-house-numbers-images	99,289	10	$32 \times 32 \times 3$ photo
GT GTSRB	harbhajansingh21/german-traffic-sign-dataset	51,839	43	$32 \times 32 \times 3$ photo
DB Diabetes	Playground Series (<code>train.csv</code>)	S5E12 300,000 700,000	of 2	24 features $\rightarrow d = 42$

4.1 SVHN

Digits cut from Google Street View photos; the label is the centre digit, neighbouring digits are noise. The two MATLAB files are loaded with `scipy.io.loadmat`, reordered from $(32, 32, 3, N)$ to $(N, 32, 32, 3)$, and label 10 is mapped to digit 0. Classes are unbalanced: digit 1 has 13,272 images, digit 9 has 4,378 (ratio 3.0). Pixels are scaled $x/255$ ($0 \rightarrow 0$, $128 \rightarrow 0.50$, $255 \rightarrow 1$); Keras uses $(N, 32, 32, 3)$, PyTorch $(N, 3, 32, 32)$.

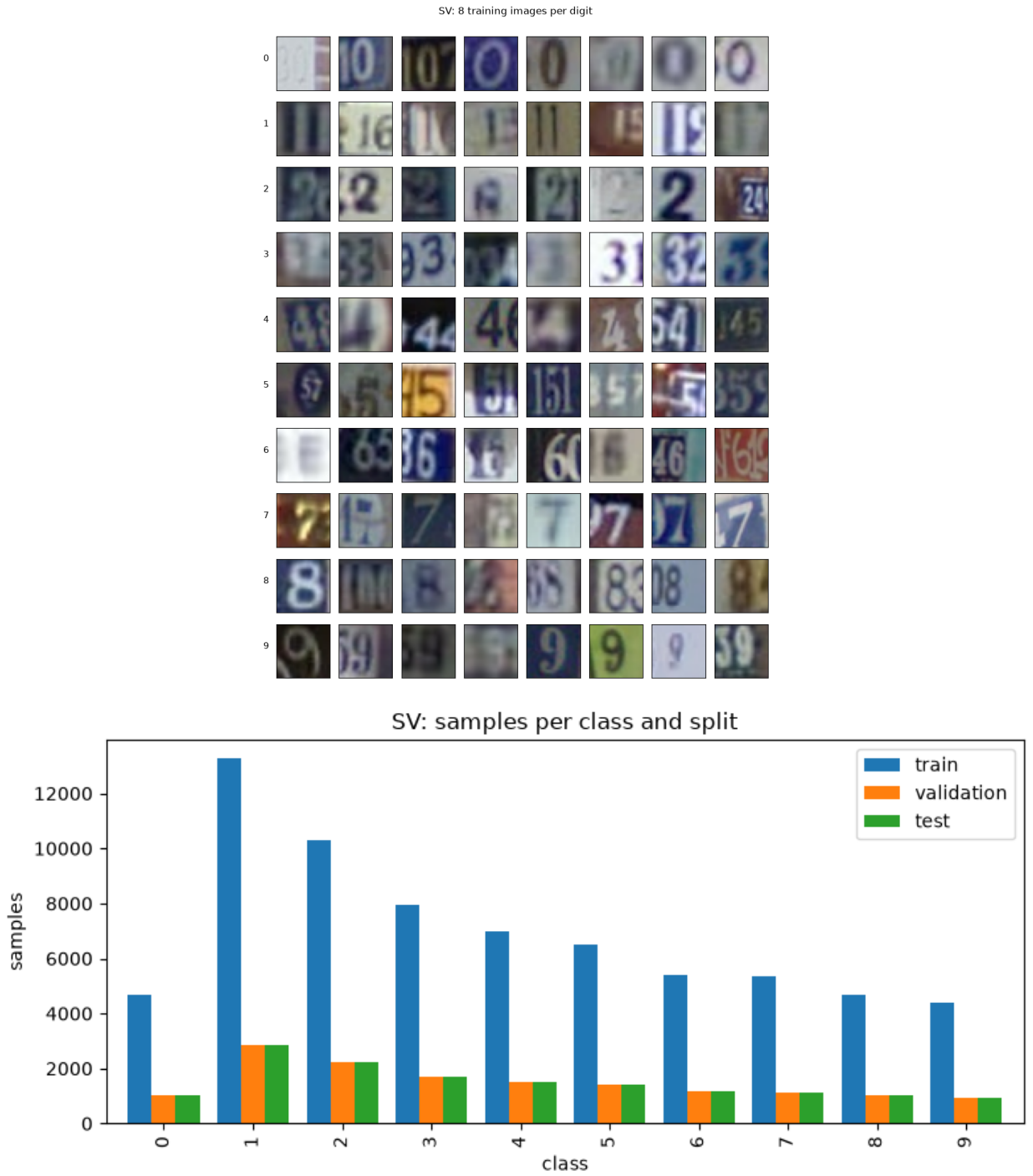


Figure 3: SV: 8 training images per digit (top) and images per class and split (bottom).

4.2 GTSRB

Traffic signs photographed from a moving car, resized to 32×32 . Three pickle files are pooled and re-split. Classes are strongly unbalanced (class 0 “Speed limit 20” has 189 images, class 2 “Speed limit 50” has 2,100; ratio 11.1), so macro F1 is the main metric. Many photos are very dark, which BatchNorm compensates for.

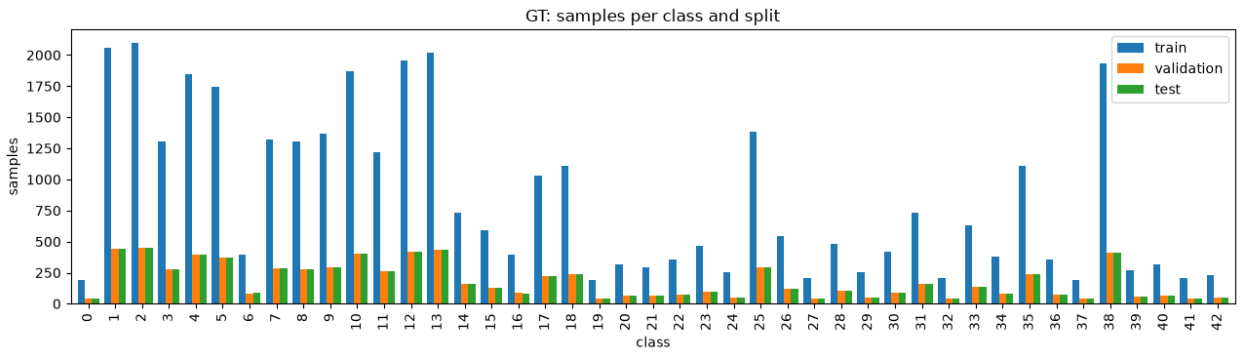
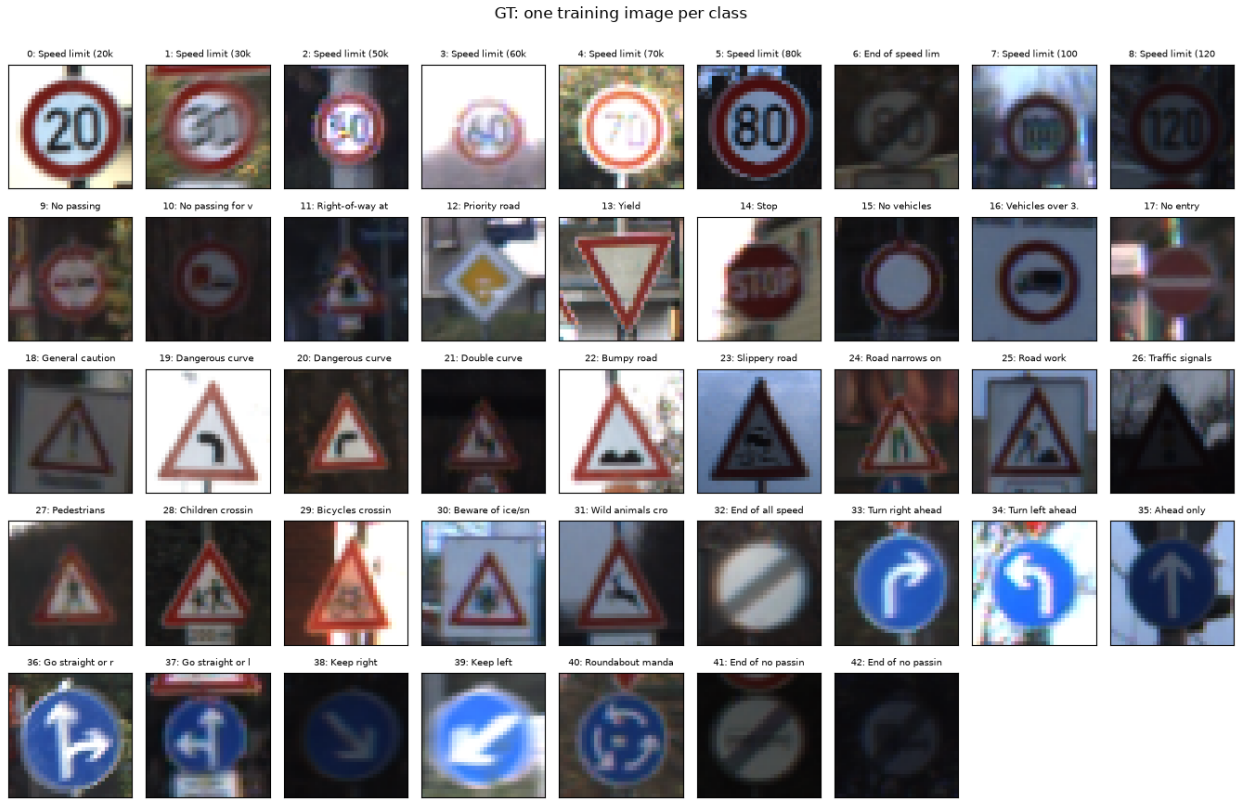


Figure 4: GT: one training image per class (top) and images per class and split (bottom).

4.3 Diabetes (Playground Series S5E12)

700,000 synthetic patients generated by Kaggle from a real dataset; a stratified sample of 300,000 is used. Each row is turned into $d = 42$ numbers: 15 numeric features standardized with training statistics ($z = (x - \mu)/\sigma$, e.g. BMI $31.7 \rightarrow (31.7 - 25.9)/2.9 = 2.0$), 6 categorical features one-hot encoded, 3 binary flags kept. 62.3% of patients are diabetic, so class weights $w_c = N/(2N_c)$ are used: $w_0 = 1.327$, $w_1 = 0.802$. The strongest numeric features differ between classes by only about 0.4 standard deviations (age, physical activity), which already signals a hard task.

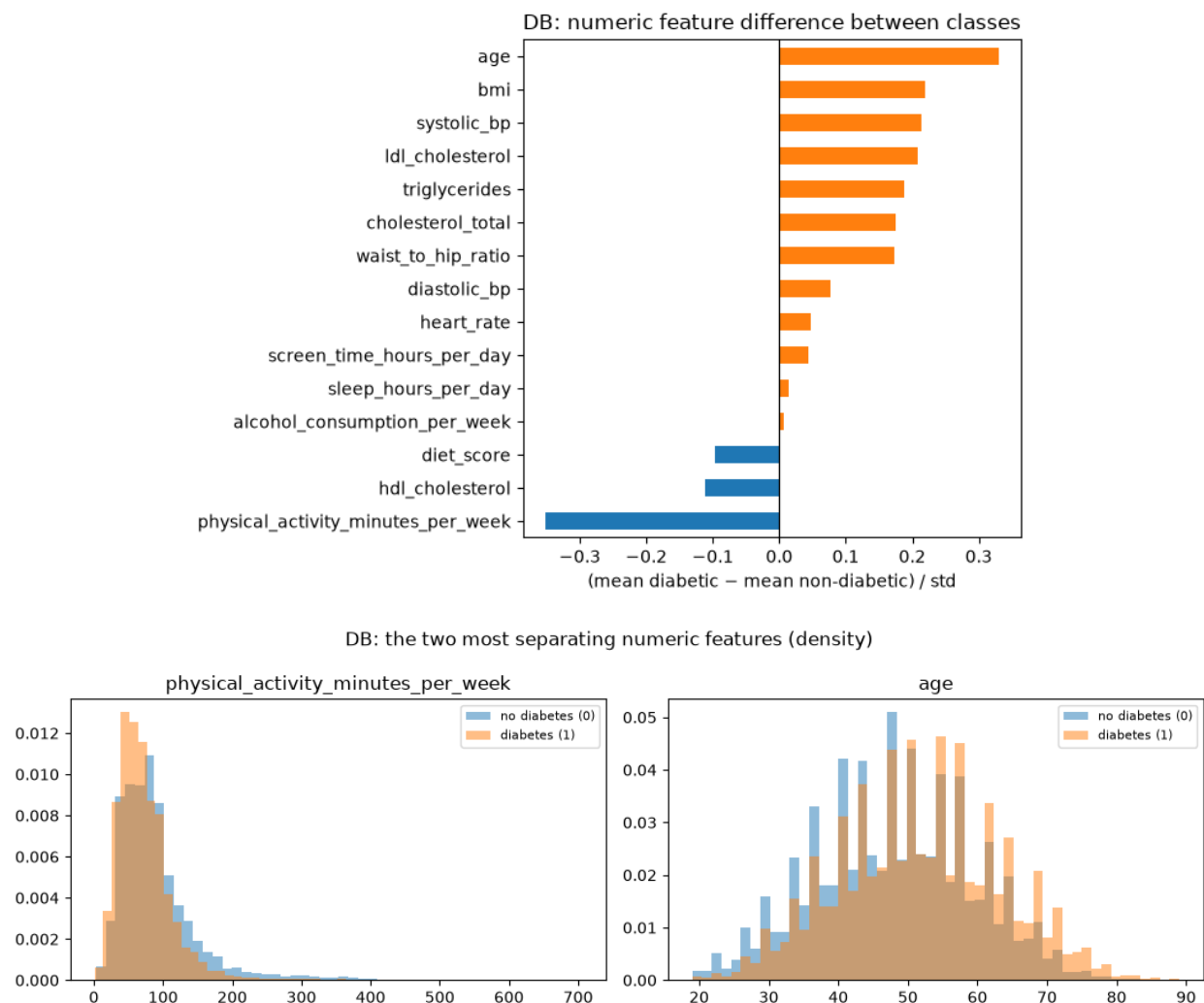


Figure 5: DB: standardized mean difference per numeric feature (top) and the two most separating features (bottom).

5 Models

Table 6: Image models (SVHN, GTSRB). 3×3 filters, padding “same”; each Conv is followed by BatchNorm and ReLU.

Step	CNN-3 output	CNN-5 output
input	$32 \times 32 \times 3$	$32 \times 32 \times 3$
block 1	Conv 32 $\rightarrow$ Pool $\rightarrow 16 \times 16 \times 32$	Conv 32 $\rightarrow$ Conv 32 $\rightarrow$ Pool $\rightarrow 16 \times 16 \times 32$
block 2	Conv 64 $\rightarrow$ Pool $\rightarrow 8 \times 8 \times 64$	Conv 64 $\rightarrow$ Conv 64 $\rightarrow$ Pool $\rightarrow 8 \times 8 \times 64$
block 3	Conv 128 $\rightarrow$ Pool $\rightarrow 4 \times 4 \times 128$	Conv 128 $\rightarrow$ Pool $\rightarrow 4 \times 4 \times 128$
head	GAP $\rightarrow$ Dropout 0.3 $\rightarrow$ Dense 256 $\rightarrow$ Dense 10/43	same
receptive field	22×22	28×28

Table 7: Tabular models (diabetes). Conv1D kernel 3, padding “same”, no pooling.

Step	CNN-3	CNN-5
Conv1D filters	$32 \rightarrow 64 \rightarrow 64$ ($d \times 64$)	$32 \rightarrow 32 \rightarrow 64 \rightarrow 64 \rightarrow 128$ ($d \times 128$)
head	Flatten $\rightarrow$ Dropout 0.3 $\rightarrow$ Dense 64 $\rightarrow$ Dense 1	same
features one position sees	7	11

Table 8: Trainable parameters (identical in TF and PT).

Model	SVHN	GTSRB	Diabetes
CNN-3	129,290	137,771	191,169
CNN-5	175,658	184,139	391,329
MLP baseline	–	–	6,977

5.1 TensorFlow, Keras and PyTorch

- **TensorFlow** (Google) is the low-level library: tensors, automatic differentiation, CPU/GPU execution.
- **Keras** is the high-level API *inside* TensorFlow: the model is declared layer by layer and trained with `compile + fit`; TensorFlow runs the computations underneath.
- **PyTorch** (Meta) is a separate library at the same level as TensorFlow.

So “TensorFlow” and “Keras” are one side of the comparison (models written in Keras, run by TensorFlow; run ID TF), and PyTorch is the other (run ID PT).

Table 9: How the same CNN is written and trained in the two frameworks.

	Keras / TensorFlow	PyTorch
model	stack layers: <code>Conv2D(32, 3, padding="same")</code>	<code>nn.Module</code> class: <code>nn.Conv2d(3, 32, 3, padding=1)</code> , input channels written by hand
image axes	$(N, 32, 32, 3)$, channels last	$(N, 3, 32, 32)$, channels first
training	<code>compile(...)</code> + <code>fit(...)</code> , one call	hand-written loop: <code>zero_grad</code> → forward → loss → <code>backward</code> → <code>step</code>
early stopping	built-in <code>EarlyStopping</code> callback	counter written inside the loop
train / test mode	switched automatically by <code>fit</code> / <code>predict</code>	<code>model.train()</code> / <code>model.eval()</code> called by hand
device	uses a GPU automatically if available	<code>.to(device)</code> for the model and every batch
strength	short, quick to write	every step visible, easy to customize
results here	same accuracy (gap < 0.011, same architecture and settings); on the CPU run Keras was 9–16% faster per epoch on images and about 50% faster on the table	

5.2 Implementation (TensorFlow/Keras)

Table 10: Keras building blocks used in the image models.

Step	Keras / TensorFlow
data	<code>tf.data.Dataset</code> : <code>shuffle</code> → <code>batch(256)</code> → divide by 255 → <code>prefetch</code> ; images $(N, 32, 32, 3)$
conv block	<code>Conv2D(F, 3, padding="same")</code> → <code>BatchNormalization(momentum=0.9)</code> → <code>ReLU()</code> → <code>MaxPooling2D(2)</code>
head	<code>GlobalAveragePooling2D()</code> → <code>Dropout(0.3)</code> → <code>Dense(256, activation="relu")</code> → <code>Dense(n_classes)</code>
init	Glorot uniform weights, zero biases (Keras default)
loss / optimizer	<code>SparseCategoricalCrossentropy(from_logits=True)</code> (diabetes: <code>BinaryCrossentropy</code>), <code>Adam(1e-3)</code>
early stopping	<code>EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)</code>

```

def build_tf(arch, input_shape, n_classes):
    inputs = keras.Input(shape=input_shape) # (32, 32, 3)
    x = inputs
    for filters, pool in arch: # e.g. [(32, True), (64, True), (128, True)]
        x = layers.Conv2D(filters, 3, padding="same")(x)
        x = layers.BatchNormalization(momentum=0.9, epsilon=1e-5)(x)
        x = layers.ReLU()(x)
        if pool:
            x = layers.MaxPooling2D(2)(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(256, activation="relu")(x)
    return keras.Model(inputs, layers.Dense(n_classes)(x)) # logits

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=20,
        callbacks=[keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
        restore_best_weights=True)])

```

One call to `fit` runs the whole training loop of Section 2.6 (batches, forward, backward, Adam update, validation, early stopping); in PyTorch these steps are written out (next subsection).

5.3 Implementation (PyTorch)

The same model in PyTorch. The layers and settings are identical to the Keras version (same parameter counts, checked in the notebooks); the training loop is written by hand.

Table 11: PyTorch building blocks used in the image models.

Step	PyTorch
data	<code>TensorDataset + DataLoader(batch_size=256, shuffle=True)</code> ; images ($N, 3, 32, 32$), divided by 255 per batch
conv block	<code>Conv2d(C_in, F, 3, padding=1) → BatchNorm2d(F) → ReLU() → MaxPool2d(2)</code>
head	<code>AdaptiveAvgPool2d(1) → Flatten() → Dropout(0.3) → Linear(128, 256) → ReLU() → Linear(256, n_classes)</code>
init	<code>xavier_uniform_ weights</code> , zero biases
loss / optimizer	<code>CrossEntropyLoss()</code> (diabetes: <code>BCEWithLogitsLoss</code>), <code>Adam(lr=1e-3)</code>
early stopping	keep the best <code>state_dict</code> ; stop after 3 epochs without a lower validation loss

```

class CNN(nn.Module):
    def __init__(self, arch, in_channels, n_classes):
        super().__init__()
        blocks, c = [], in_channels
        for filters, pool in arch: # e.g. [(32, True), (64, True), (128, True)]
            blocks += [nn.Conv2d(c, filters, 3, padding=1), nn.BatchNorm2d(filters), nn.ReLU()]
            if pool:
                blocks.append(nn.MaxPool2d(2))
            c = filters
        self.features = nn.Sequential(*blocks)
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(0.3),
                                   nn.Linear(c, 256), nn.ReLU(), nn.Linear(256, n_classes))

    def forward(self, x):
        return self.head(self.features(x))

for x, y in train_loader: # one epoch
    optimizer.zero_grad() # clear gradients
    loss = criterion(model(x), y) # forward + cross-entropy
    loss.backward() # backpropagation
    optimizer.step() # Adam update

```

6 Results per Dataset

How to read the figures

- **Training curves** (loss and accuracy per epoch, train vs. validation). Both losses falling = still learning. Training loss falling while validation loss rises = **overfitting**; early stopping keeps the epoch before that.
- **Confusion matrix** (row-normalized). Row = true class, column = predicted class; each row sums to 1. The diagonal is the recall of each class. Example (Figure 7): row 7, column 1 = 0.06 means 6% of the real 7s were predicted as 1.
- **Feature maps**: grayscale images of one filter's output; bright pixels = where that filter's pattern was found.
- **ROC curve** (diabetes): true-positive rate against false-positive rate for every threshold; the diagonal is guessing (AUC 0.5), the top-left corner is perfect (AUC 1).

6.1 SVHN

Table 12: SV test results.

Run ID	Accuracy	Macro F1	Epochs	s/epoch	Train time (s)	ms / 1k pred.
SV-TF-CNN-3	0.9117	0.9072	20	45.0	898	0.10
SV-TF-CNN-5	0.9306	0.9265	10	86.5	868	0.22
SV-PT-CNN-3	0.9106	0.9046	18	48.9	884	0.27
SV-PT-CNN-5	0.9294	0.9250	13	98.7	1,287	0.49

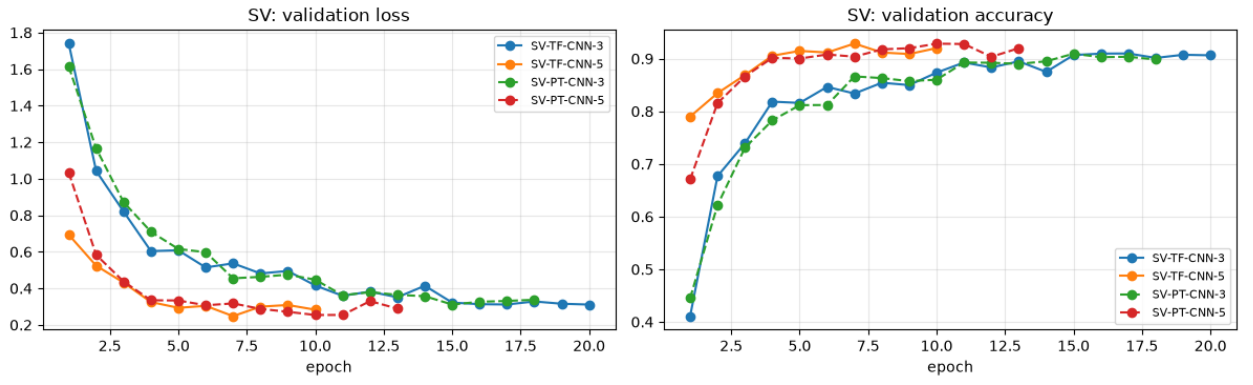


Figure 6: SV: validation loss and accuracy of the four runs (solid = TF, dashed = PT).

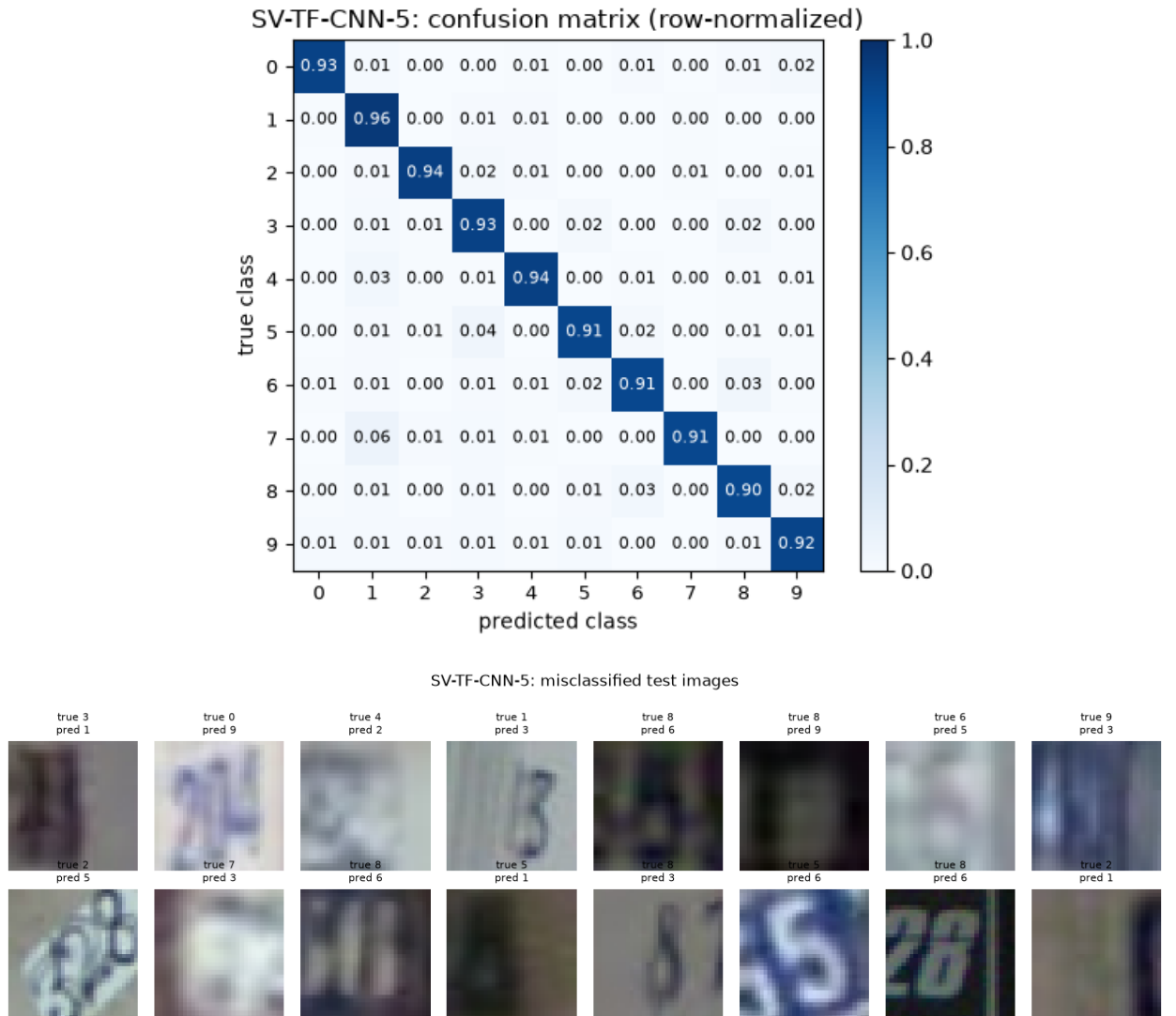


Figure 7: SV-TF-CNN-5: confusion matrix (top) and misclassified test images (bottom).

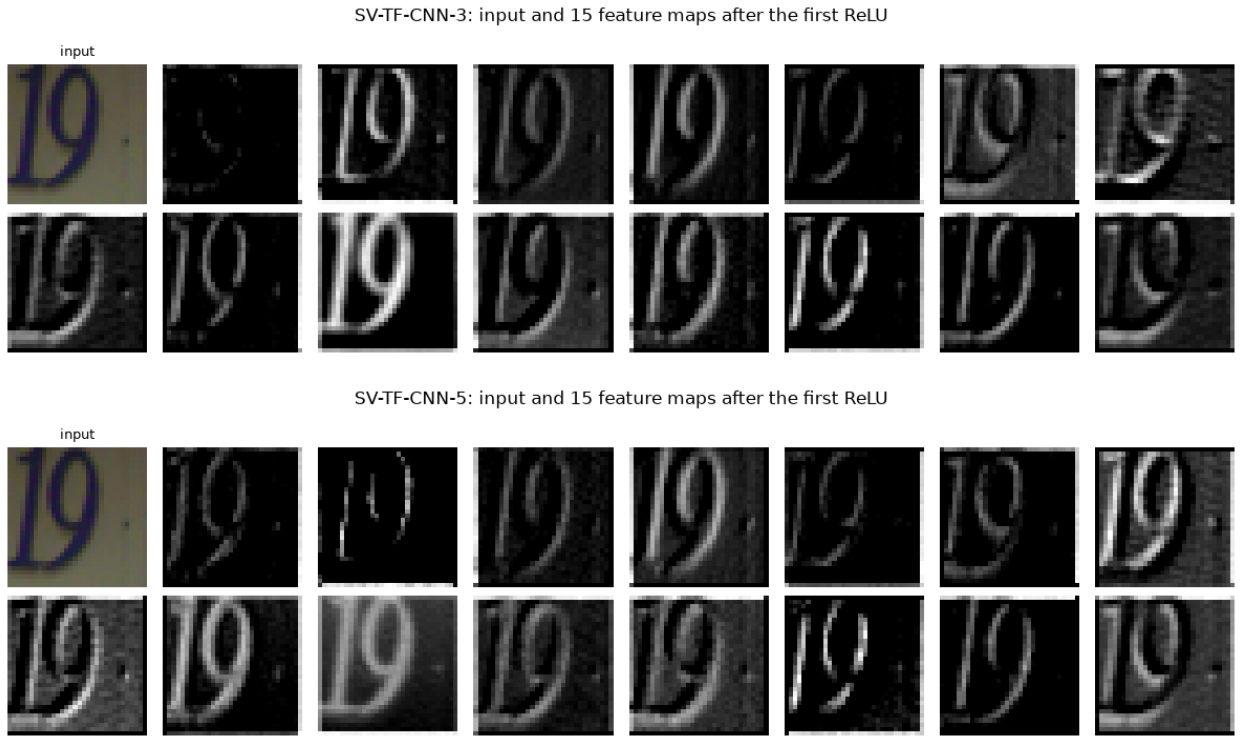


Figure 8: First-layer feature maps after ReLU for one test image: SV-TF-CNN-3 (top), SV-TF-CNN-5 (bottom).

- **Depth:** CNN-5 improves accuracy by +0.019 in both frameworks and macro F1 by +0.019 (TF) / +0.020 (PT), and reaches its best validation loss in fewer epochs (10 and 13 vs. 20 and 18), at about $2\times$ the time per epoch.
- **Framework:** TF and PT differ by 0.001 accuracy for both depths.
- **Errors:** the most frequent confusion is $7 \rightarrow 1$ (6% of the 7s), then $5 \rightarrow 3$ (4%) and $4 \rightarrow 1$ (3%): digits that share a vertical stroke. The misclassified grid shows the typical hard cases (blur, low contrast, neighbouring digits).
- The first-layer feature maps respond to the digit's strokes and edges; a few maps respond to background colour.

6.2 GTSRB

Table 13: GT test results.

Run ID	Accuracy	Macro F1	Epochs	s/epoch	Train time (s)	ms / 1k pred.
GT-TF-CNN-3	0.9781	0.9751	20	23.3	470	0.12
GT-TF-CNN-5	0.9911	0.9907	11	45.5	507	0.22
GT-PT-CNN-3	0.9848	0.9843	20	25.9	518	0.28
GT-PT-CNN-5	0.9943	0.9937	16	52.8	846	0.50

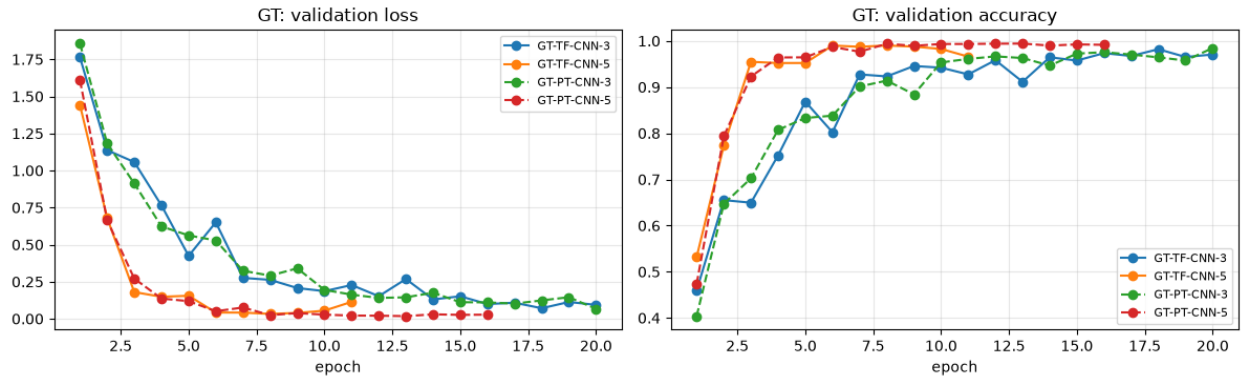


Figure 9: GT: validation loss and accuracy of the four runs (solid = TF, dashed = PT).

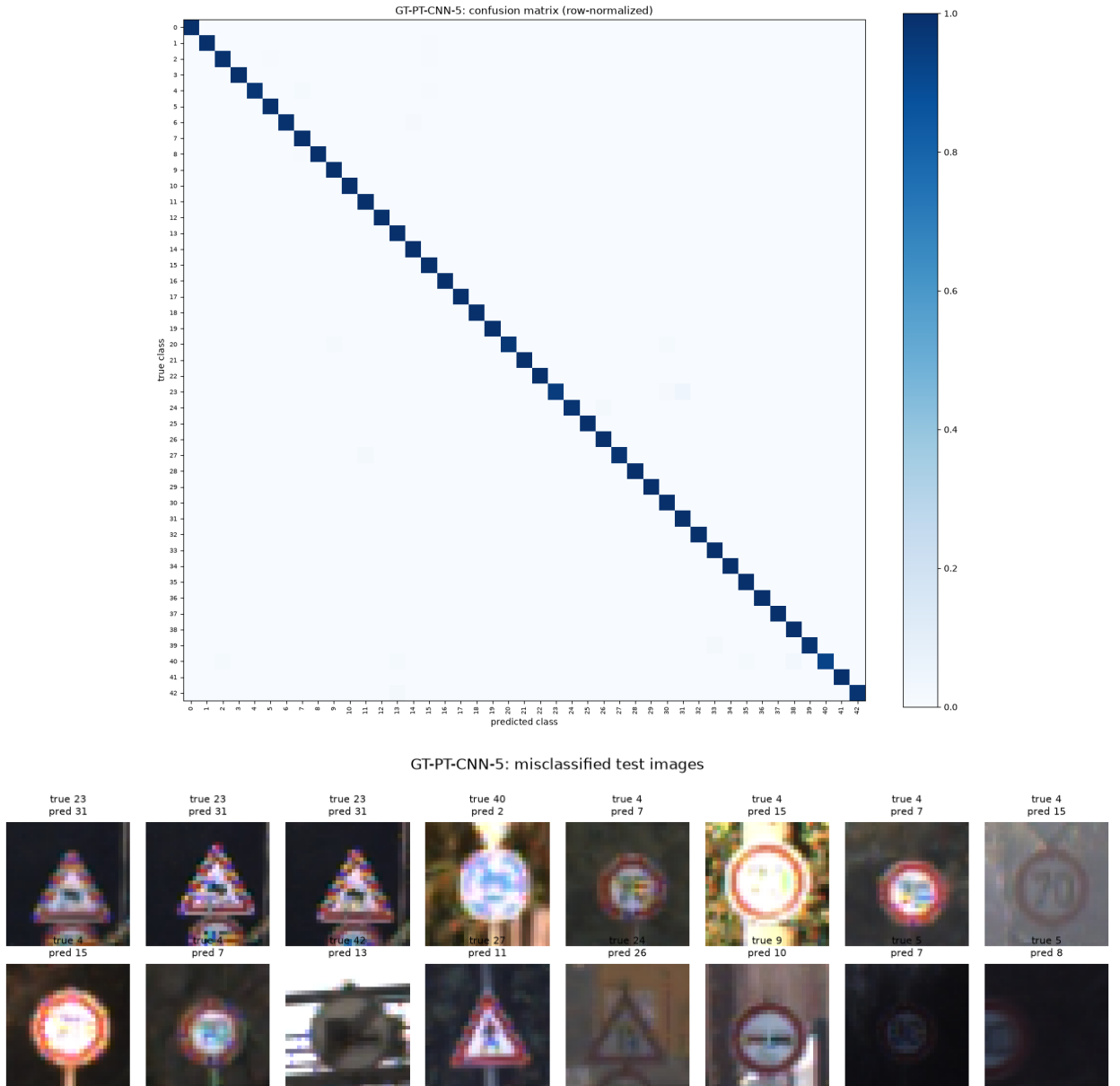


Figure 10: GT-PT-CNN-5: confusion matrix over 43 classes (top) and misclassified test signs (bottom).

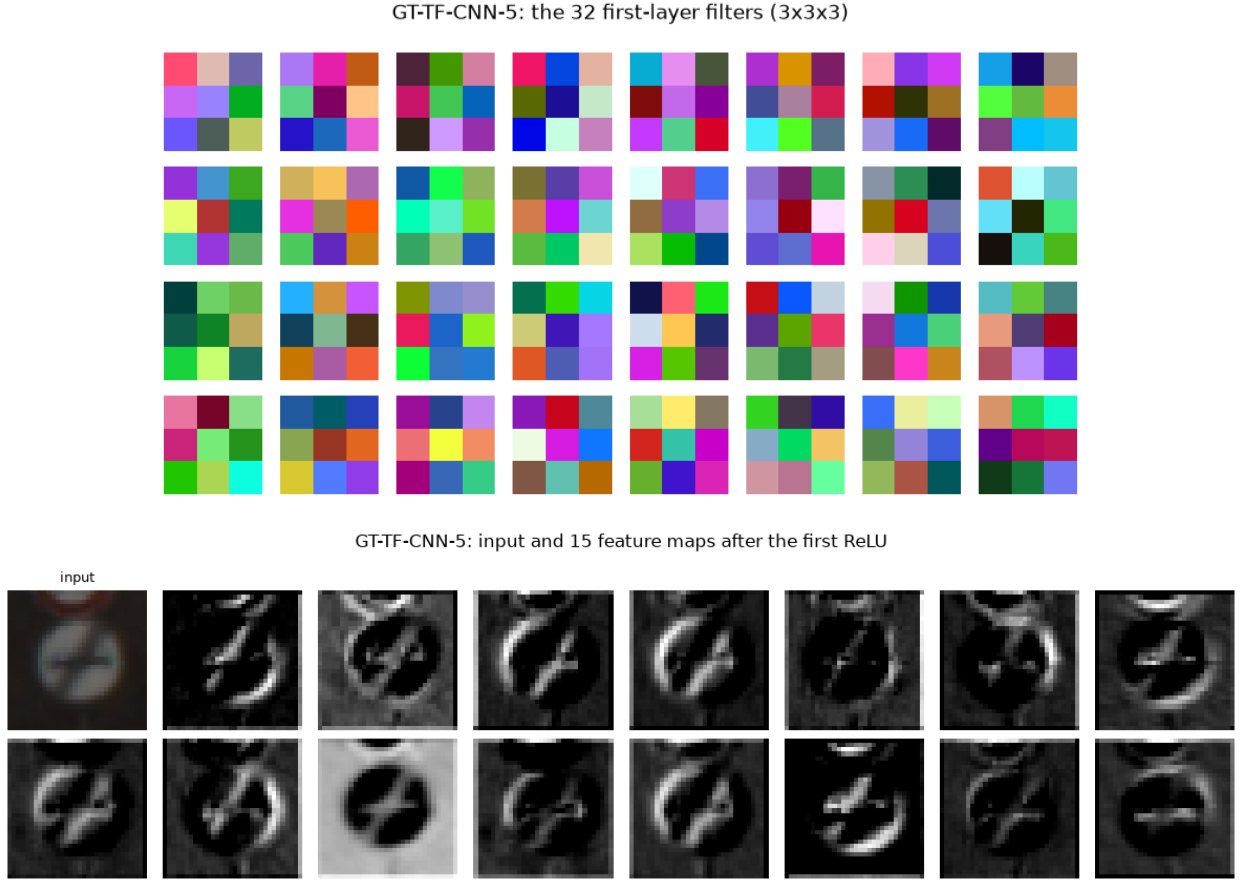


Figure 11: GT-TF-CNN-5: the 32 first-layer filters (top) and feature maps for one test sign (bottom).

- **Depth:** CNN-5 raises macro F1 by +0.016 (TF) and +0.009 (PT). Because macro F1 weights the 43 classes equally, this gain includes the rare signs, not only the frequent speed limits.
- **Framework:** PT is 0.003–0.007 higher in accuracy. Both CNN-3 runs used all 20 epochs (the best TF epoch was the last one), so CNN-3 had not fully converged; the small TF–PT gap is within that training noise.
- The confusion matrix is almost purely diagonal; the misclassified grid shows the remaining hard cases (dark or blurred photos).

6.3 Diabetes

Table 14: DB test results (threshold 0.5).

Run ID	Accuracy	Macro F1	F1 (pos.)	ROC-AUC	Params	Epochs	s/epoch
DB-TF-CNN-3	0.6465	0.6310	0.7067	0.6964	191,169	7	12.8
DB-TF-CNN-5	0.6333	0.6259	0.6785	0.6938	391,329	5	28.8
DB-PT-CNN-3	0.6371	0.6280	0.6862	0.6963	191,169	7	20.1
DB-PT-CNN-5	0.6446	0.6308	0.7024	0.6957	391,329	8	42.5
DB-TF-MLP (baseline)	0.6275	0.6222	0.6672	0.6948	6,977	5	1.1

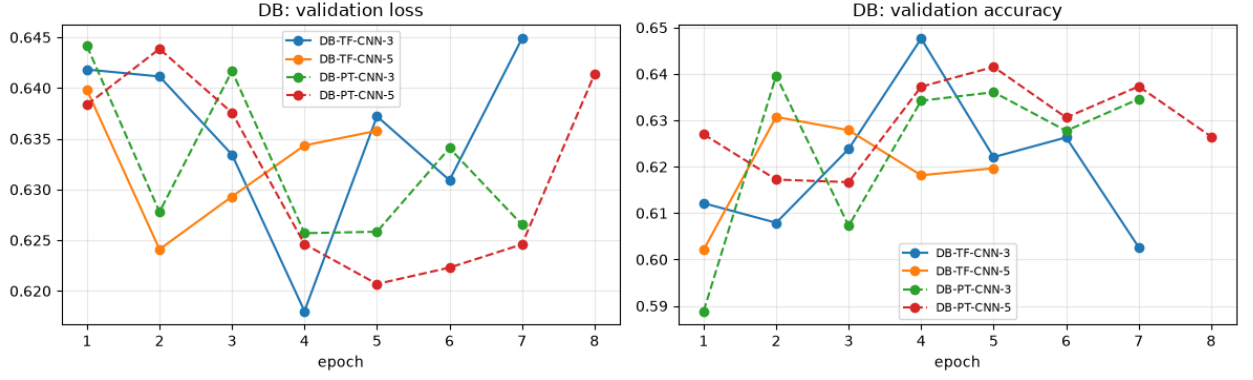


Figure 12: DB: validation loss and accuracy of the four runs (solid = TF, dashed = PT).

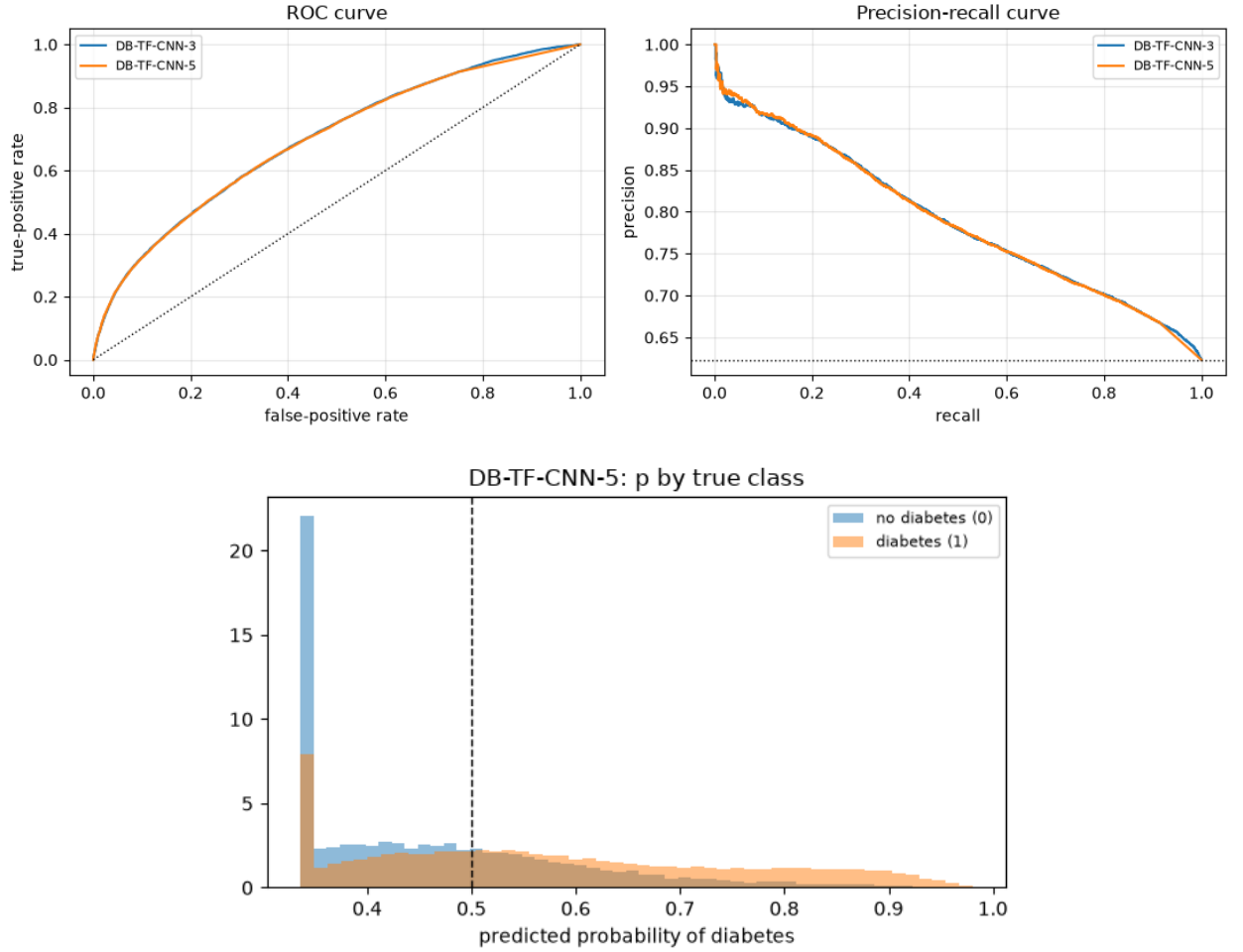


Figure 13: DB-TF: ROC and precision-recall curves (top); predicted probability by true class for DB-TF-CNN-5 (bottom).

- **Depth:** no gain. ROC-AUC changes by -0.003 (TF) and -0.001 (PT) from CNN-3 to CNN-5, while parameters double and time per epoch grows $2.1\text{--}2.2\times$.
- **Framework:** TF and PT are within 0.002 ROC-AUC; accuracy differences (± 0.01) come from where predictions fall around the 0.5 threshold.
- **Baseline:** the MLP reaches ROC-AUC 0.6948 with $56\times$ fewer parameters. The convolution’s “neighbouring features” assumption adds nothing here.

- The validation curves zigzag in a narrow band (loss 0.62–0.645): with a weak signal, the models reach a noisy optimum within 2–5 epochs and early stopping ends training soon after.
- The probability histogram shows the two classes overlap strongly; a threshold other than 0.5 would trade precision for recall but not separate them.

7 Cross-Dataset Comparison

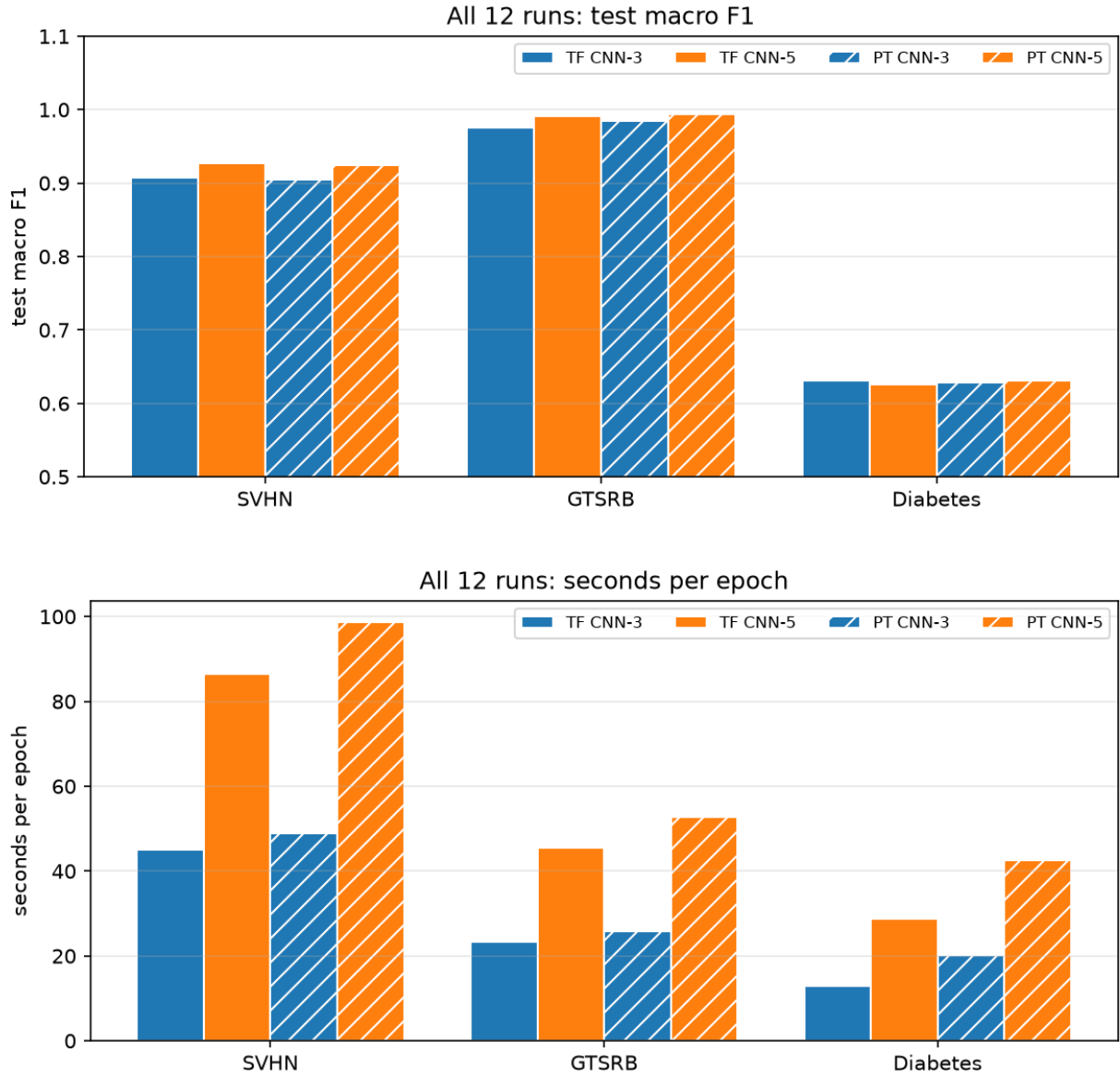


Figure 14: All 12 runs: test macro F1 (top) and training time per epoch (bottom). Blue = CNN-3, orange = CNN-5, hatched = PyTorch.

Table 15: Effect of depth (CNN-5 minus CNN-3) and of framework (PT minus TF).

Comparison		SVHN	GTSRB	Diabetes
CNN-5 – CNN-3, macro F1	TF	+0.0194	+0.0156	−0.0051
	PT	+0.0204	+0.0094	+0.0028
CNN-5 / CNN-3, time per epoch	TF	1.92×	1.95×	2.24×
	PT	2.02×	2.04×	2.11×
PT – TF, macro F1	CNN-3	−0.0025	+0.0092	−0.0030
	CNN-5	−0.0015	+0.0029	+0.0048
PT / TF, time per epoch	CNN-3	1.09×	1.11×	1.57×
	CNN-5	1.14×	1.16×	1.48×

- **Depth** pays off where the input has spatial structure: +0.01 to +0.02 macro F1 on both image datasets. On the table it changes nothing, because the information is limited by the features, and Conv1D has no spatial structure to exploit.
- **Frameworks** agree in quality: every PT–TF gap is below 0.01 macro F1, the same size as the run-to-run noise of a single seed. They differ in speed: on this CPU run Keras was faster per epoch everywhere, and much faster on the small tabular model, where PyTorch’s per-batch Python loop overhead matters most. Keras also predicted 2.3–2.7× faster per 1,000 images (1.3× on the table).
- **Cost:** CNN-5 takes about 2× the time per epoch, but it often stops earlier (fewer epochs), so its total training time is only 0.97–1.6× that of CNN-3 on the image datasets.

7.1 Keras vs. PyTorch outputs

The same model trained in both frameworks, side by side (test set).

Dataset	Model	Keras / TensorFlow				PyTorch			
		Acc.	Macro F1	Epochs	s/epoch	Acc.	Macro F1	Epochs	s/epoch
SVHN	CNN-3	0.9117	0.9072	20	45.0	0.9106	0.9046	18	48.9
SVHN	CNN-5	0.9306	0.9265	10	86.5	0.9294	0.9250	13	98.7
GTSRB	CNN-3	0.9781	0.9751	20	23.3	0.9848	0.9843	20	25.9
GTSRB	CNN-5	0.9911	0.9907	11	45.5	0.9943	0.9937	16	52.8
Diabetes	CNN-3	0.6465	0.6310	7	12.8	0.6371	0.6280	7	20.1
Diabetes	CNN-5	0.6333	0.6259	5	28.8	0.6446	0.6308	8	42.5

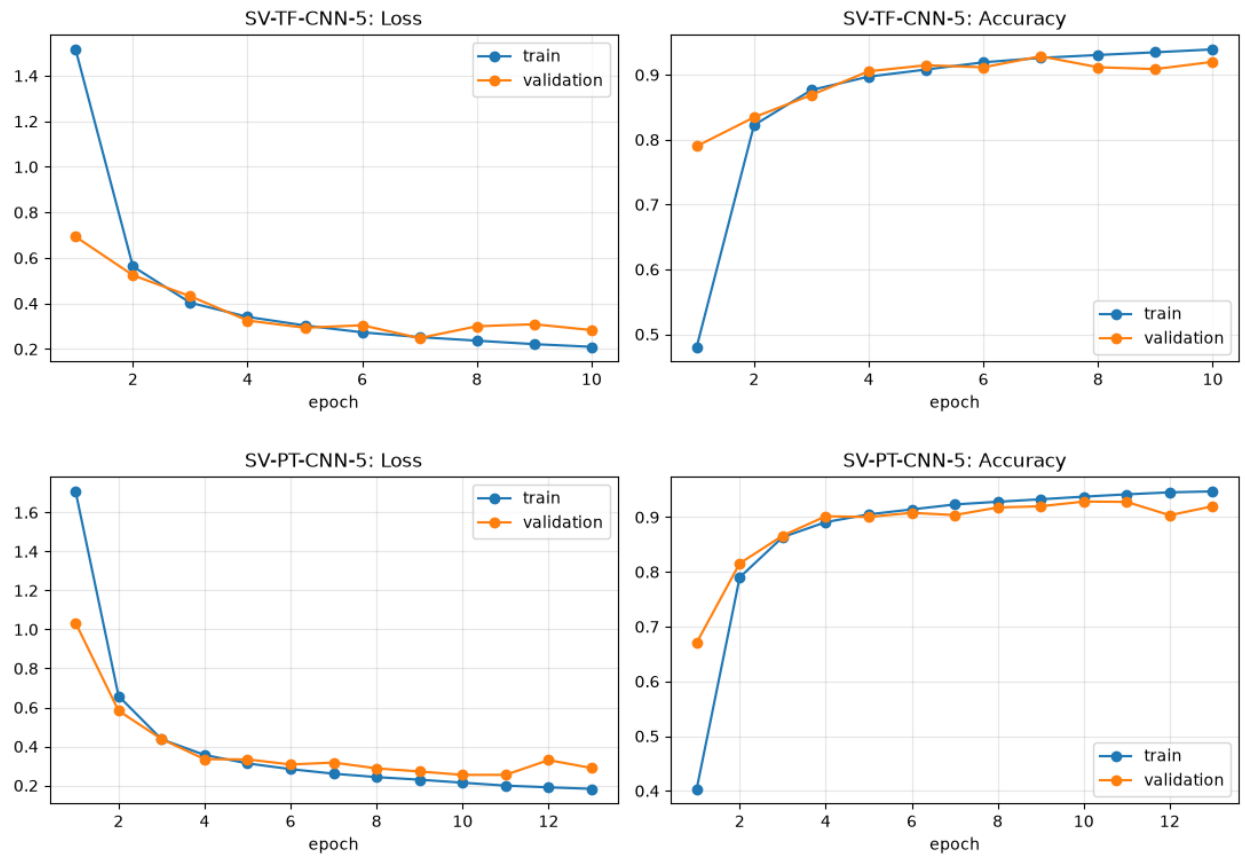


Figure 15: Training curves of CNN-5 on SVHN: SV-TF-CNN-5 (top) and SV-PT-CNN-5 (bottom).

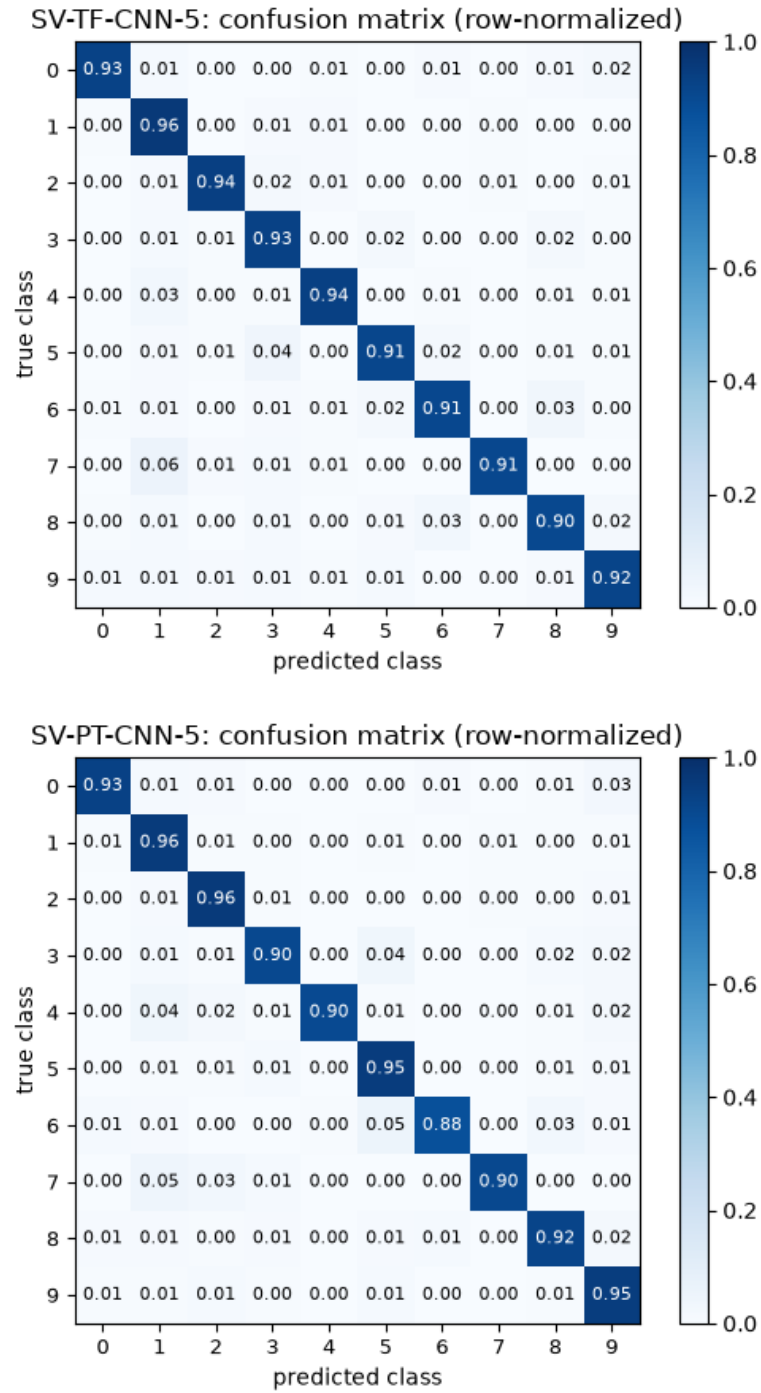


Figure 16: Confusion matrices of CNN-5 on SVHN: SV-TF-CNN-5 (top) and SV-PT-CNN-5 (bottom).

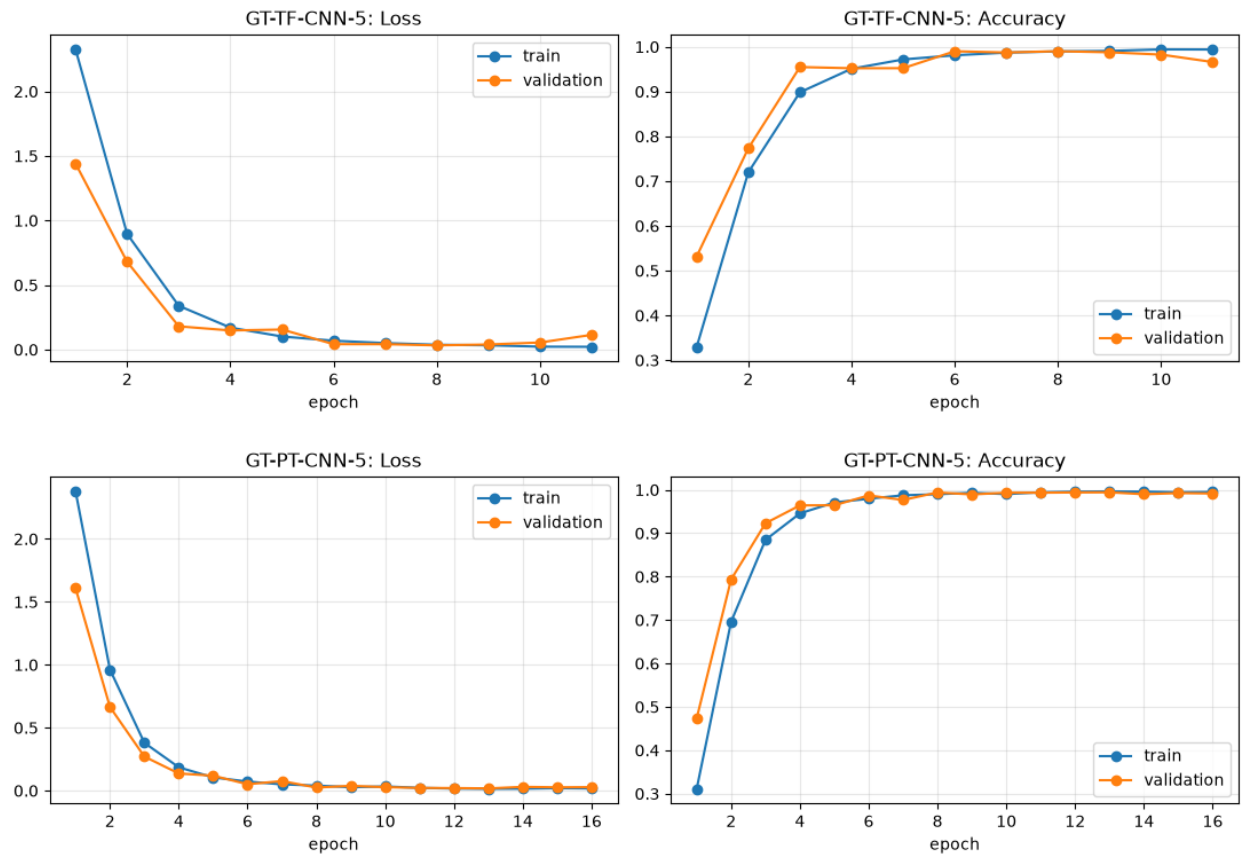


Figure 17: Training curves of CNN-5 on GTSRB: GT-TF-CNN-5 (top) and GT-PT-CNN-5 (bottom).

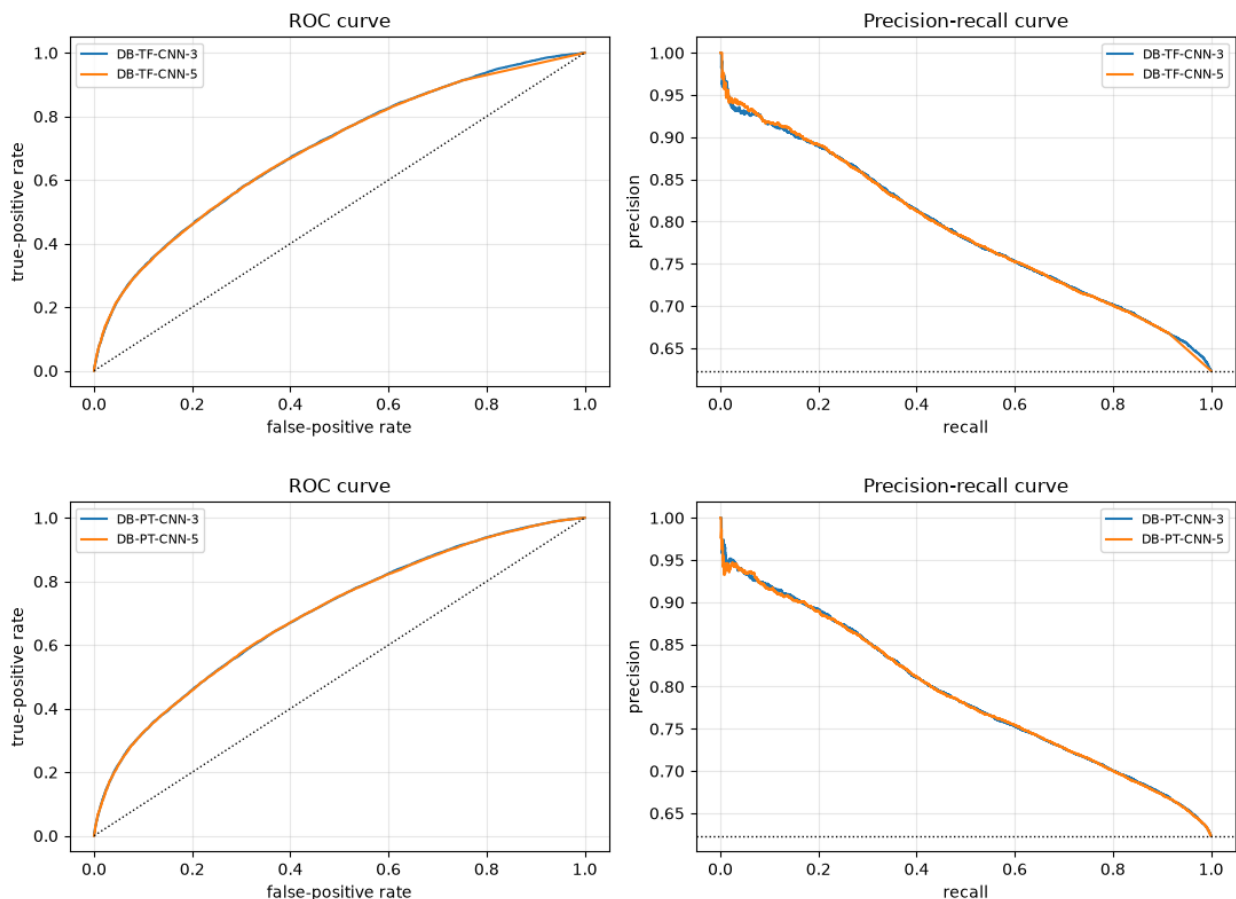


Figure 18: ROC and precision-recall curves on diabetes: Keras models (top) and PyTorch models (bottom).

7.2 Why depth helps on images: the receptive field

Each 3×3 convolution lets a unit see one more pixel on every side, and each pooling doubles the step between neighbouring units. The **receptive field** (input area one unit can see) therefore grows as $RF \leftarrow RF + (K - 1) \cdot j$, with the jump j doubling after each pooling:

CNN-3	conv1	+pool	conv2	+pool	conv3	+pool		
receptive field	3	4	8	10	18	22		
CNN-5	conv1	conv2	+pool	conv3	conv4	+pool	conv5	+pool
receptive field	3	5	6	10	14	16	24	28

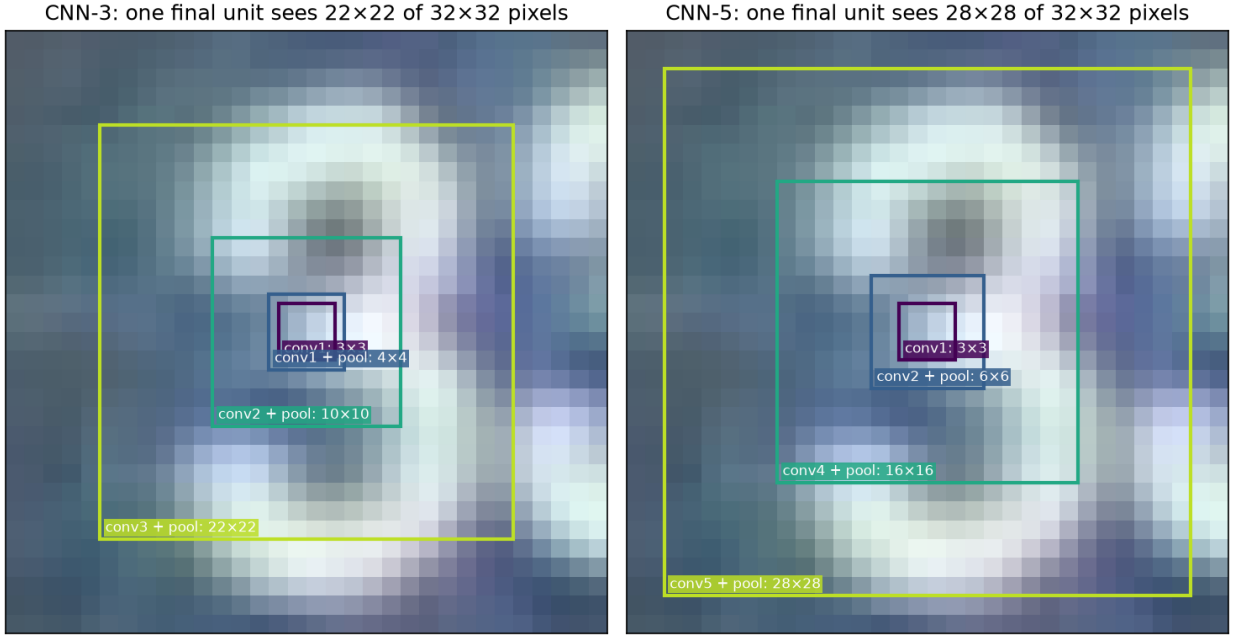


Figure 19: Receptive field after each block, drawn on an SVHN image. A final CNN-3 unit sees 22×22 pixels; a CNN-5 unit sees 28×28 , almost the whole digit and its neighbours.

A CNN-5 unit can therefore relate the top and the bottom of a digit (e.g. the open or closed left side that separates a 3 from a 5 or an 8), and it has two more non-linear layers to combine patterns. That matches the results: $+0.02$ macro F1 on SVHN, where digits fill the image and have look-alikes, and $+0.009$ to $+0.016$ on GTSRB.

7.3 Why a CNN does not help on the diabetes table

	Image (SVHN, GTSRB)	Table (diabetes)
neighbours	pixels next to each other belong to the same stroke or edge	columns next to each other are unrelated (e.g. <code>screen_time</code> next to <code>bmi</code>); the order was chosen by us
same pattern, other place	an edge means the same anywhere, so one filter can be reused everywhere	position = feature; a pattern at “age” means something else at “cholesterol”
signal in the data	strong: models reach 0.93–0.99	weak: the best feature differs between classes by only ≈ 0.4 standard deviations
result	CNN-5 > CNN-3	CNN-3 $\approx$ CNN-5 $\approx$ MLP (ROC-AUC 0.694–0.696)

The two assumptions that make convolutions efficient on images (local neighbours matter, the same filter works everywhere) do not hold for a table. Adding depth or filters then only adds parameters; the ceiling is set by the information in the 24 features, which every model reaches after 2–5 epochs.

8 Conclusion and Limitations

1. Two extra convolutional layers (CNN-5) improve SVHN and GTSRB by 1–2 macro-F1 points, with a larger receptive field (28×28 vs. 22×22) at about twice the cost per epoch.
2. On the diabetes table, depth and convolutions do not help: four CNNs and a 7k-parameter MLP all reach ROC-AUC ≈ 0.695 . For tabular data a CNN is not the natural model.

3. The same architecture gives the same result in TensorFlow/Keras and PyTorch (gaps < 0.01), confirming that the frameworks differ in abstraction level, not in the learning method.
4. Keras is shorter to write and was faster on this CPU; PyTorch exposes every training step and is easier to customize.

Limitations. One seed per run (no variance estimate); GPU/CPU kernels are not fully deterministic; no data augmentation; the diabetes data is synthetic; Conv1D depends on an arbitrary column order; timing comes from one CPU-only Windows machine; two CNN-3 image runs reached the 20-epoch limit.

A Appendix

A.1 Additional figures

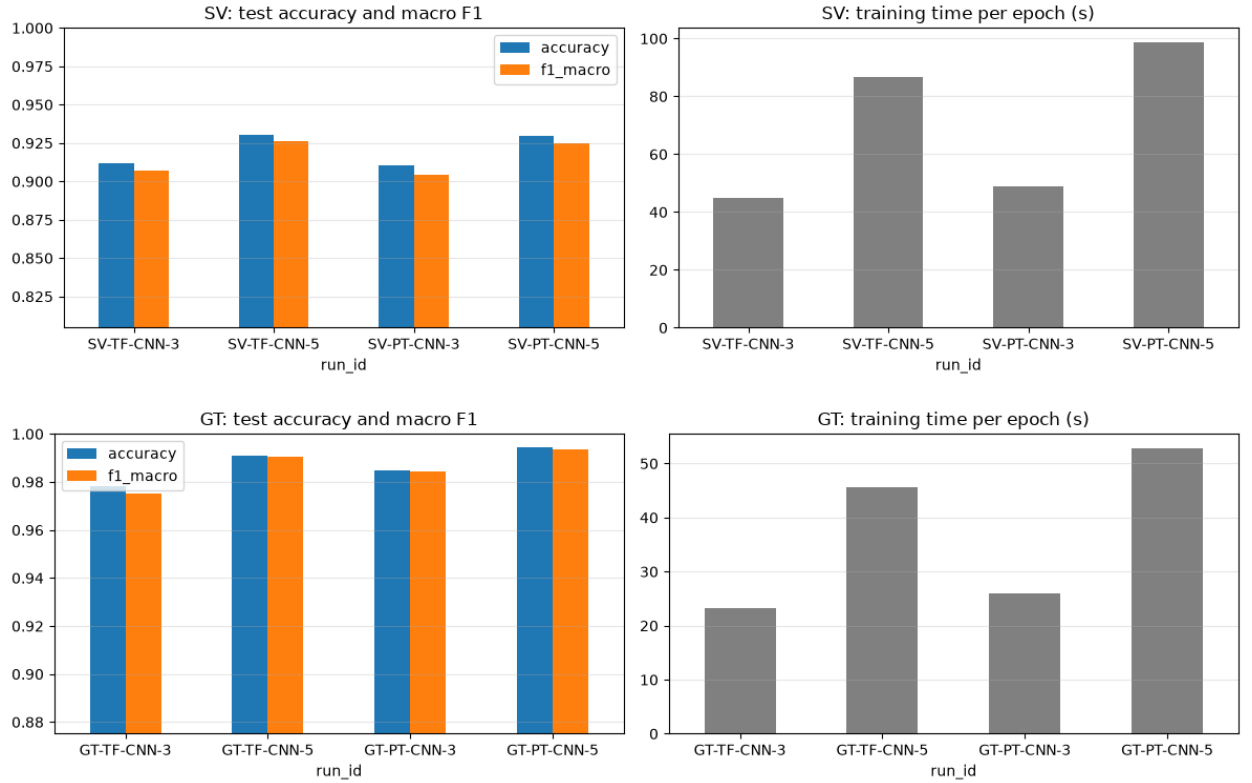


Figure 20: Per-dataset metric and time bars: SV (top), GT (bottom).

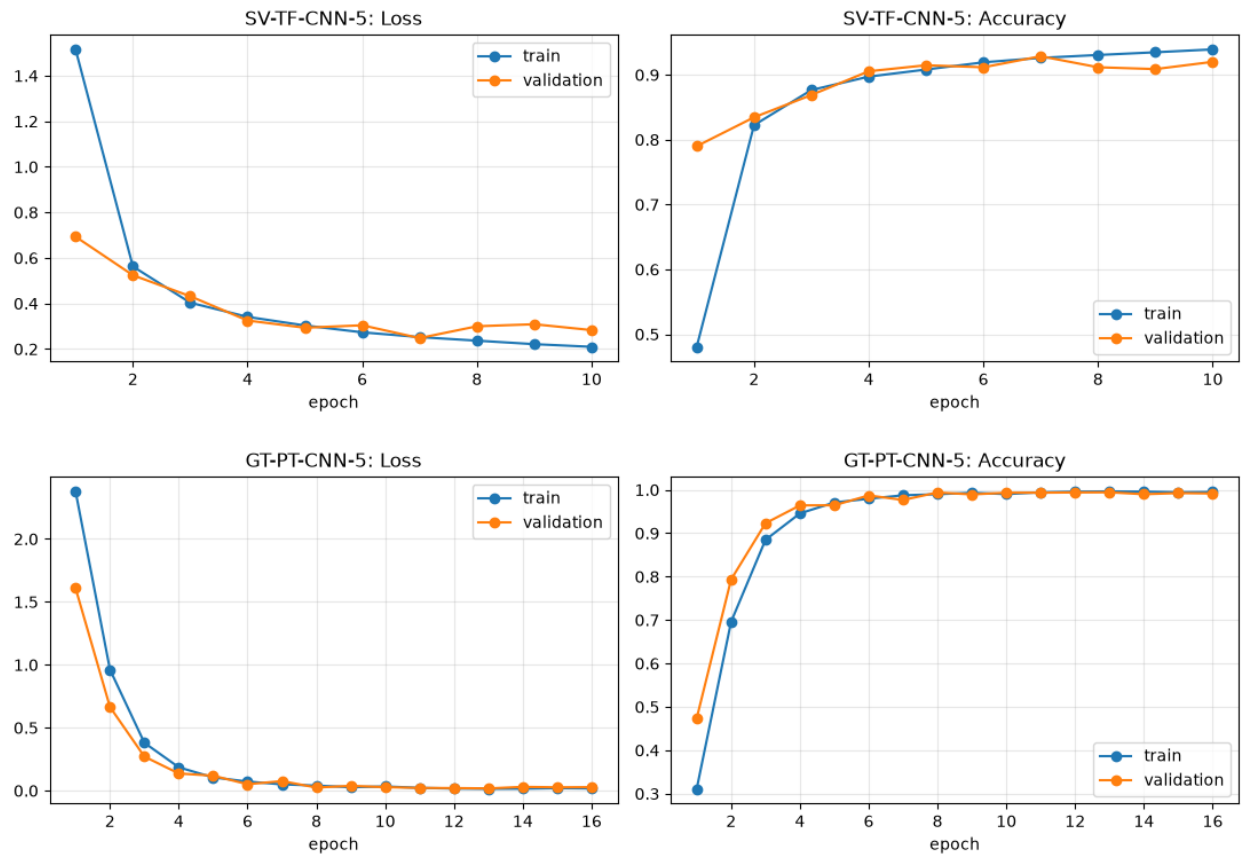
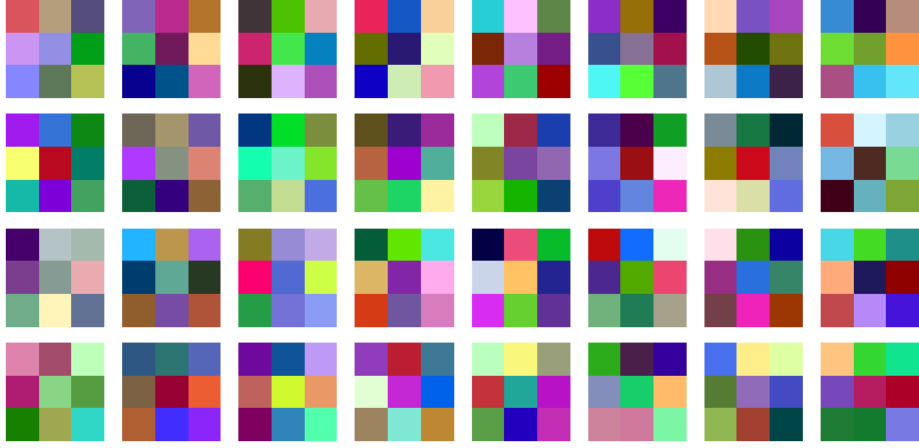


Figure 21: Training curves of the best image models: SV-TF-CNN-5 (top), GT-PT-CNN-5 (bottom).

SV-TF-CNN-5: the 32 first-layer filters (3x3x3)



SV-PT-CNN-5: the 32 first-layer filters (3x3x3)

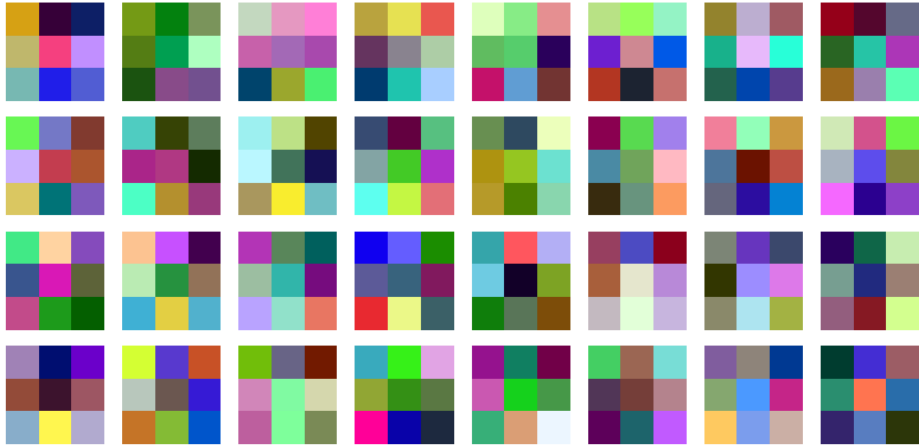


Figure 22: First-layer filters: SV-TF-CNN-5 (top), SV-PT-CNN-5 (bottom).

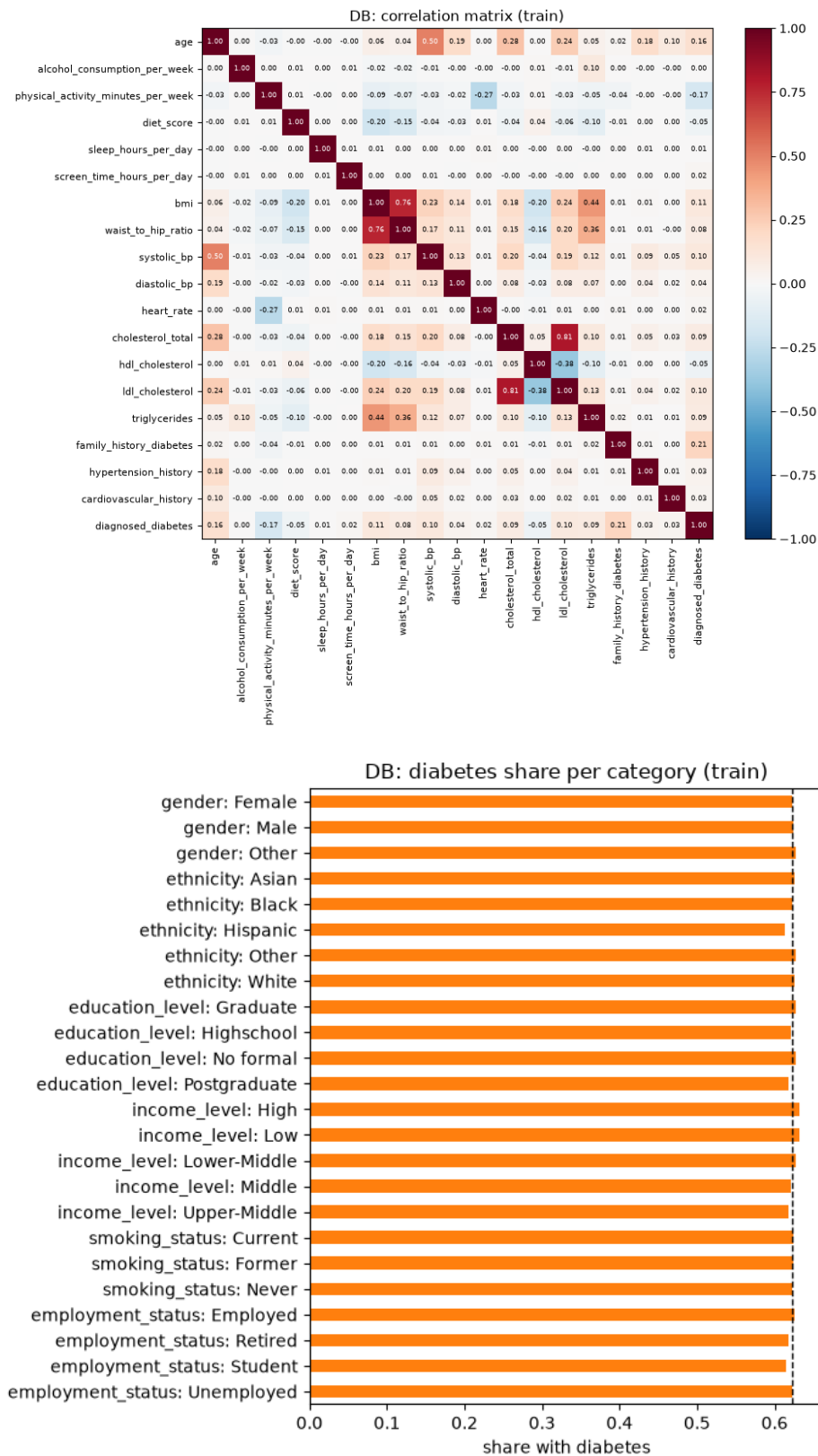


Figure 23: DB: correlation matrix (top) and diabetes share per category (bottom).

A.2 How to run the project

1. Environment (Windows, macOS or Linux; Python 3.10–3.12):

```
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # macOS / Linux
```

```
pip install -r requirements.txt
```

GPU: Apple Silicon uses `tensorflow-metal` and `mps` automatically; on an NVIDIA GPU PyTorch uses `cuda` (on Windows install the CUDA build first, see below); TensorFlow uses an NVIDIA GPU only on Linux/WSL2. Without a GPU both run on the CPU.

```
pip install torch --index-url https://download.pytorch.org/whl/cu126
```

2. Data. Download from Kaggle and place the files as follows (see `dataset/README.md`):

Folder	Files
dataset/sv/	train_32x32.mat, test_32x32.mat
dataset/gt/	train.p, valid.p, test.p, signname.csv
dataset/db/	train.csv (competition playground-series-s5e12; join it first)

3. Run the notebooks. Open `sv_svhn.ipynb`, `gt_gtsrb.ipynb` or `db_diabetes.ipynb` in `notebook/` with Jupyter or VS Code and run all cells, or from `notebook/`:

```
jupyter nbconvert --to notebook --execute --inplace db_diabetes.ipynb
```

Each notebook is self-contained. On the first run it creates `dataset/<code>/splits.npz` (the 70/15/15 split), then trains the four models and writes `notebook/results/<run_id>.json`, the saved models in `notebook/models/` and the figures in `report/images/`. Start with `db_diabetes` (fastest).